딥러닝 출석수업

인간과 기계가 함께 배우는 세상

딥러닝의 기초부터 응용까지

조현준

딥러닝 출석수업

학습목차

01 딥러닝의 기본 이해(신경망과 역전파)

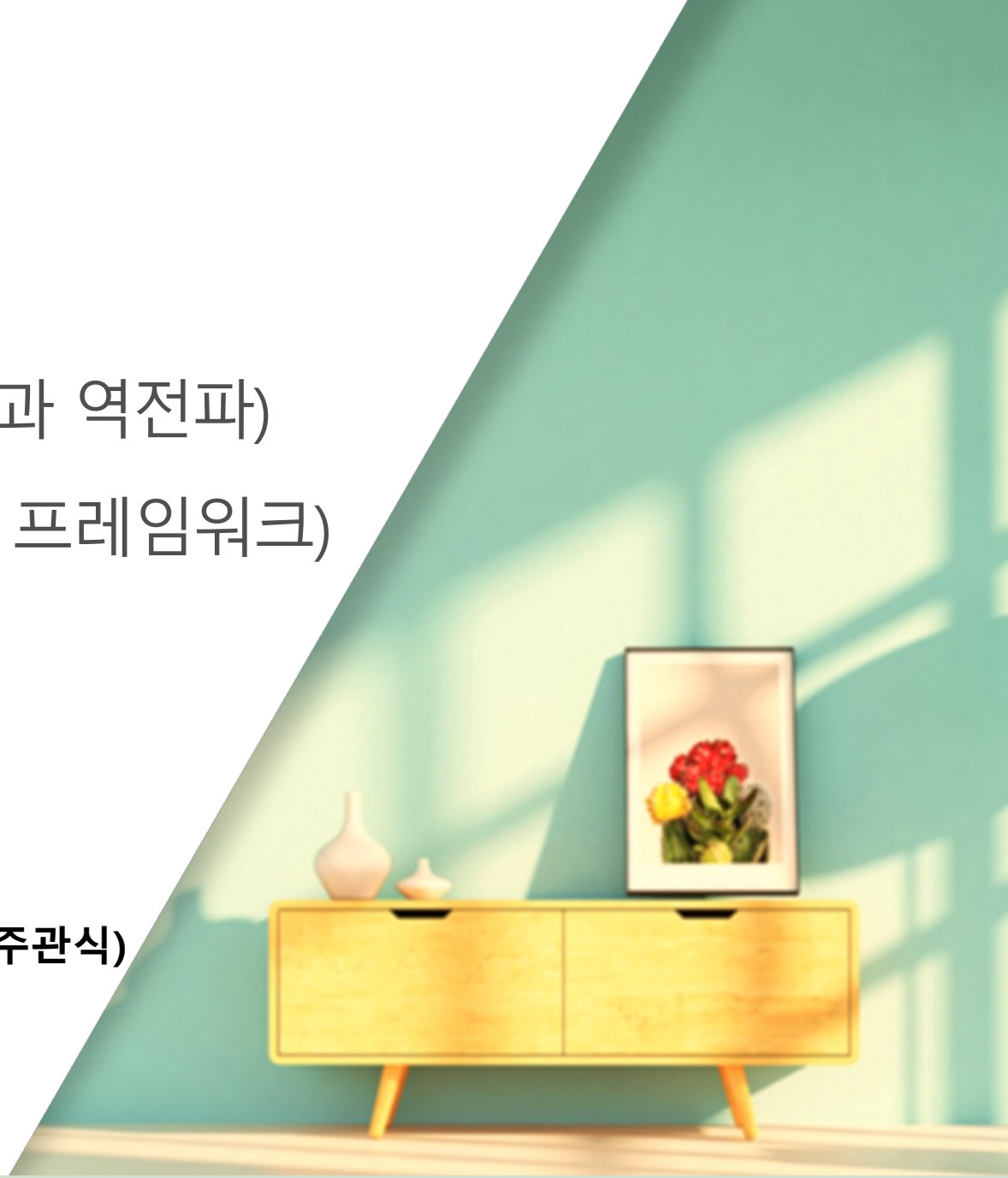
02 딥러닝의 기본 구현(딥러닝 프레임워크)

총 점수 :30점

출석 : 15점

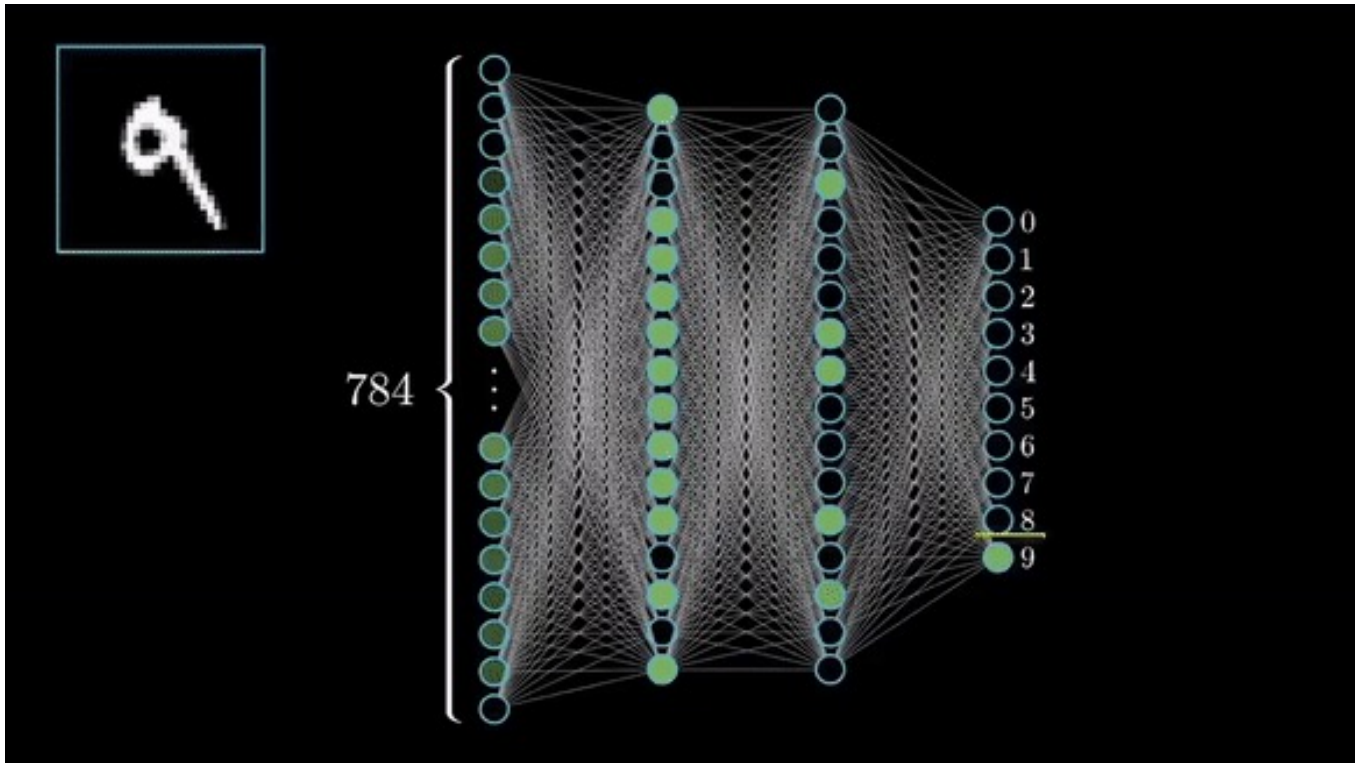
과제 : 15점

- 오늘 배운 내용 기반 **딥러닝 과제(객관식+주관식)**

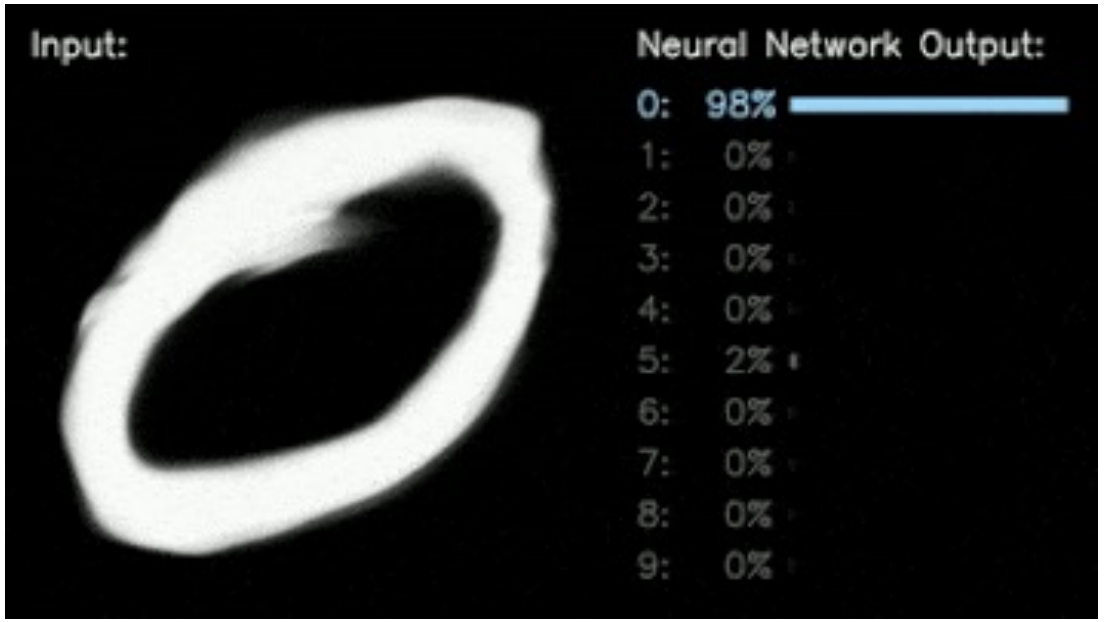


그 외 딥러닝 사례 : 숫자 손글씨 인식 기술

딥러닝을 활용한 '9'손글씨의 예측과정



딥러닝을 활용한 0~9까지 손글씨 정확도





딥러닝의 기본 정의란?



딥러닝의 기본 정의란?

데이터로부터 특정 패턴을 찾기 위해서 인공 신경망을 활용한 것

https://designer.microsoft.com/

디지털 데이터 세상





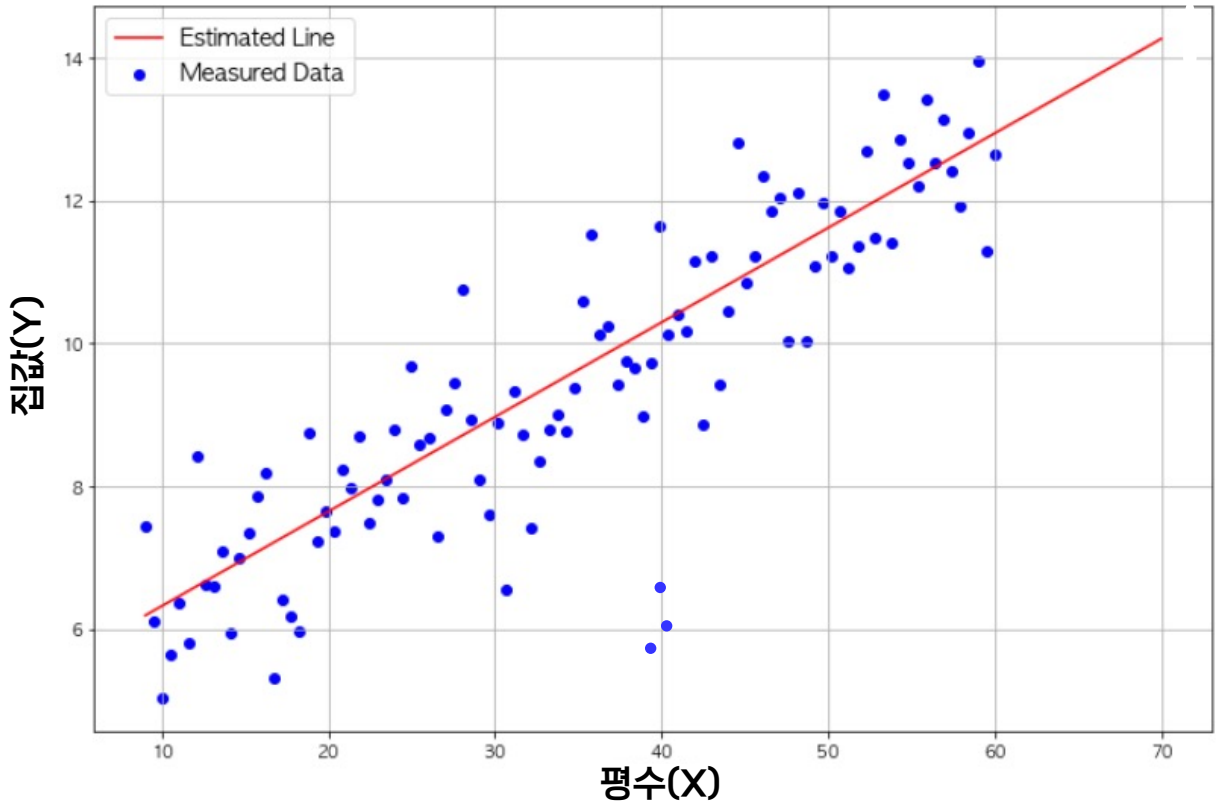
일상 생활 대부분 정량(수치 or 디지털)적 데이터로 활용 가능

(데이터를 이용한 특정 패턴을 찾을 수 있는 기반이 마련됨)

데이터로부터의 특정 패턴 찾기 : 학습 과정(Training)

특정 패턴 : 입력이 주어졌을 때, 그에 딱 맞는 결과(예측)를 내주는 함수

특정 패턴 예시: 평수(X)에 따른 집 값(Y) 데이터

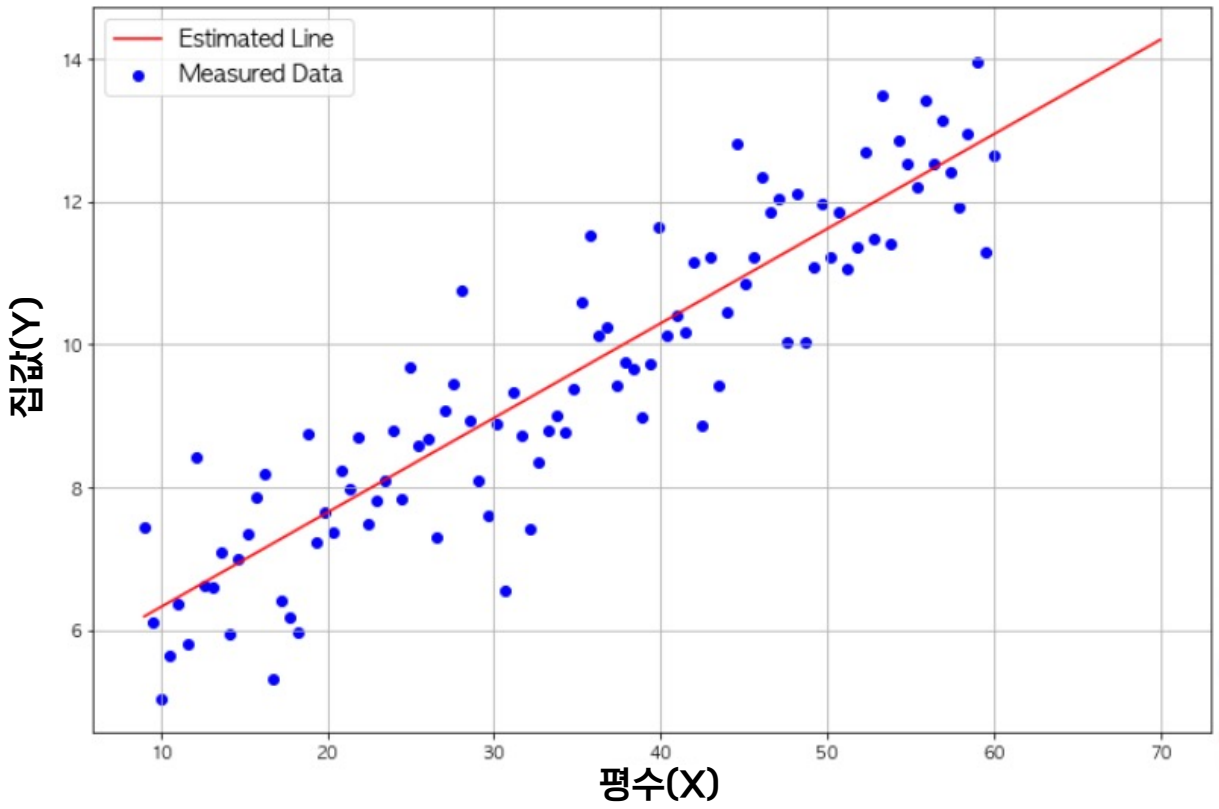


패턴 찾는 과정

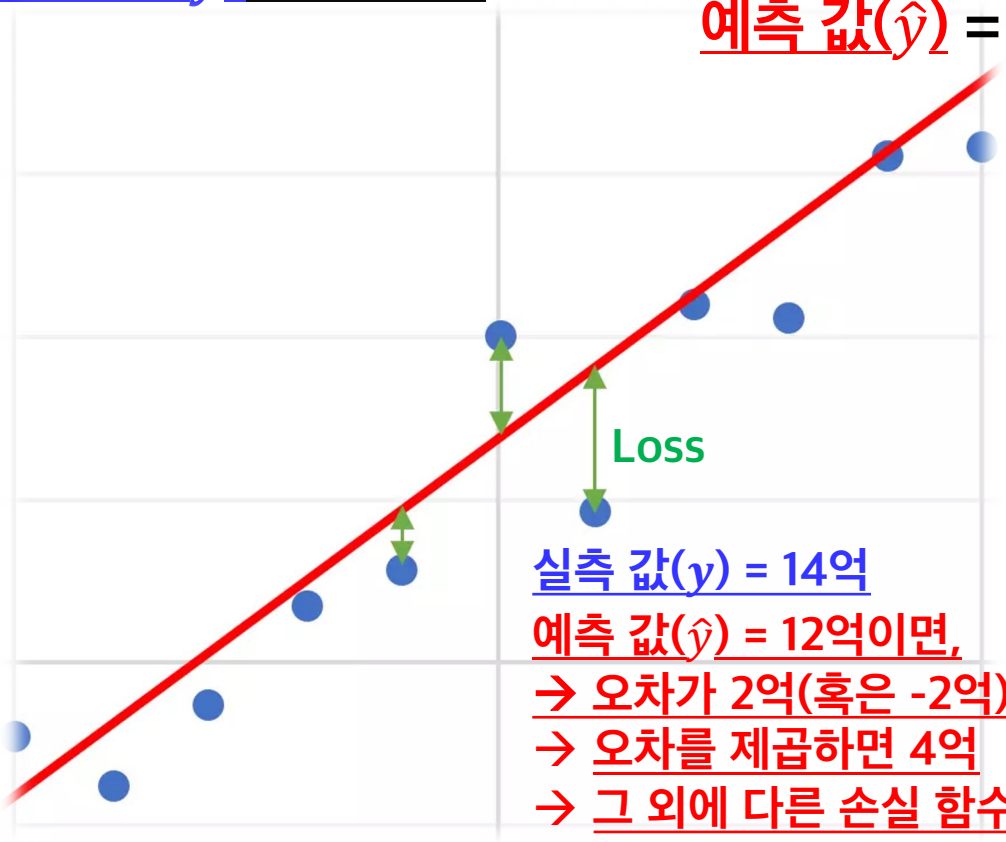
1. 실측 데이터 회득(Blue Pt)
2. 실측 데이터 정제 및 분석
3. 직선 추정(Red Line) 반복 🔄
   Y = aX + b
   최적의 a와 b를 찾는 과정 → 학습(Training)
   최적의 매개 변수(parameter)를 찾는 과정
   → a = weight
   → b = bias

Loss Function(or Cost Function)

손실 값(Loss or Cost or Error)은 예측 값($\hat{y}$) 과 실측 값(y)과 차이 값



예측 값($\hat{y}$) = wX + b



실측 값(y) = 14억

예측 값($\hat{y}$) = 12억이면,

→ 오차가 2억(혹은 -2억)

→ 오차를 제곱하면 4억

→ 그 외에 다른 손실 함수도 많은 편

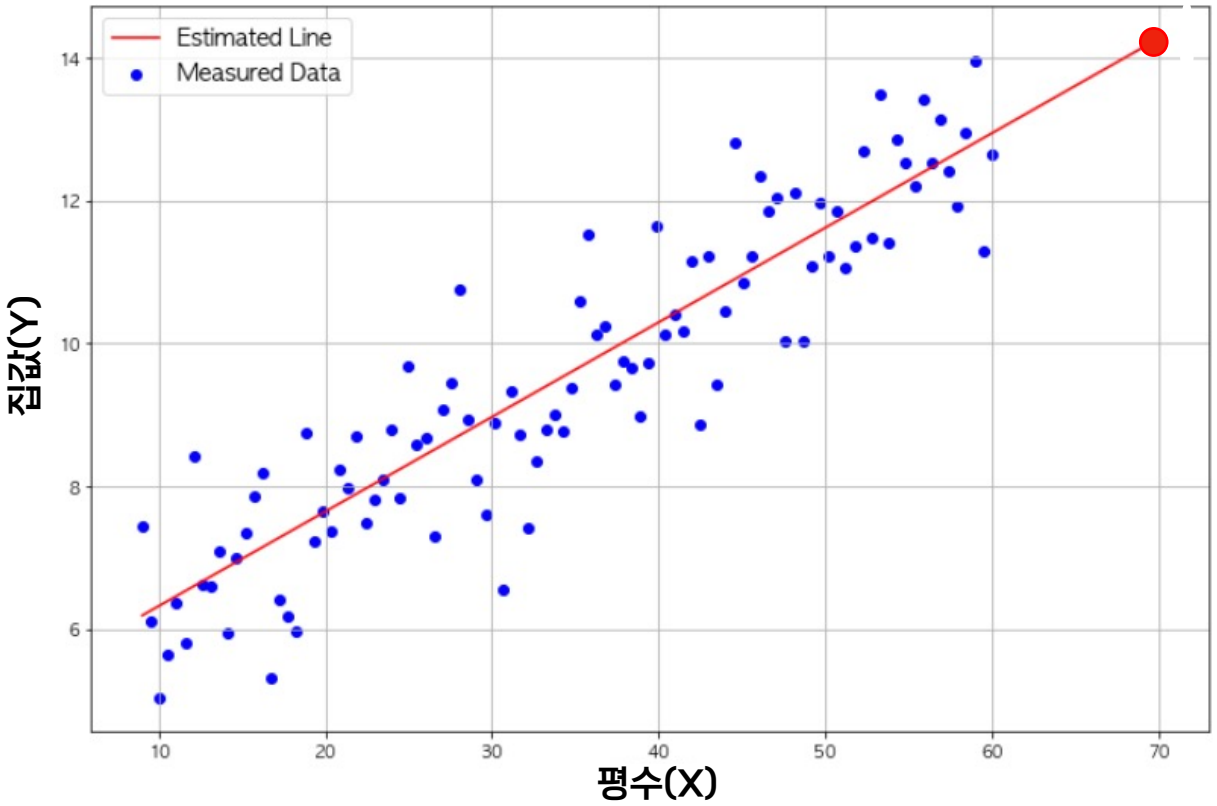
Mean Squared Error $= \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - (wx_i + b))^2$$

예측(Prediction)

(계산된 파라미터 a,b가 적용) 학습된 모델을 활용하여 원하는 결과를 예측

추론 예시: 68평수에 해당하는 집값은?



모델함수의 **입력 값(평수)**을 적용하여 원하는 **결과(집값)**을 얻을 수 있음

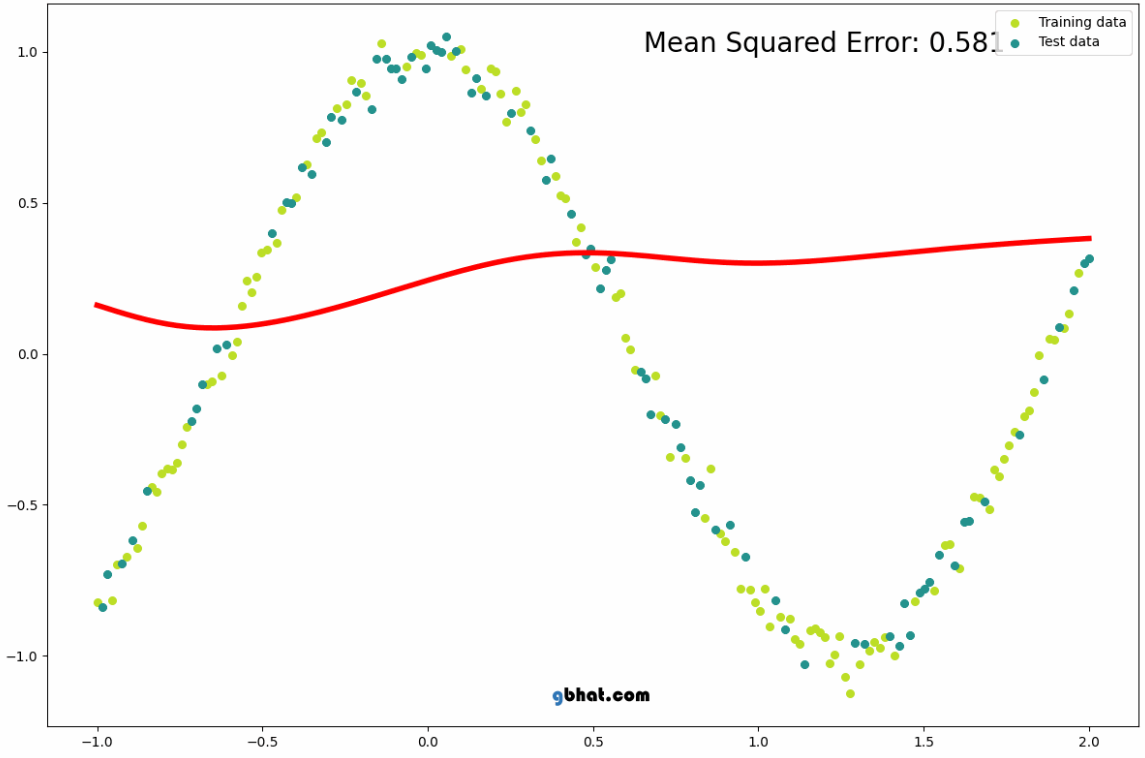
$Y = aX + b$

14억 = 0.1323529*68 + 5

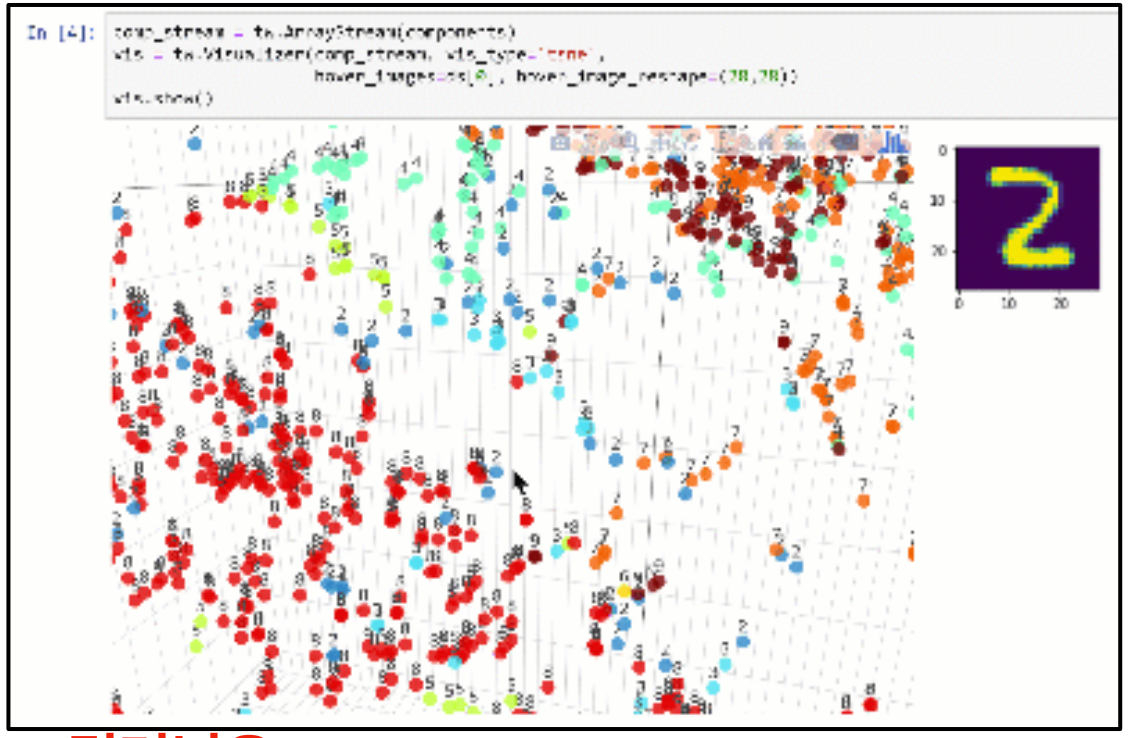
Model Complexity

모델 복잡도가 커질수록 보다 정밀한 결과를 예측 → 파라미터 개수가 늘어남

1차~고차(6차) 모델 추정 과정



딥러닝 데이터 시각화 예시



→ 딥러닝은
휠씬 더 복잡한 패턴에 대한 추정이 가능

인공 신경망이란?



1. 인공 신경망

인공 신경망(artificial neural networks)이란?

- 인간과 동물의 두뇌를 구성하는 생물학적 신경 시스템(뉴런)의 원리를 바탕으로 설계된 계산 시스템



1. Function
뉴런은 입력 신호를 받아서 출력 신호를 생성

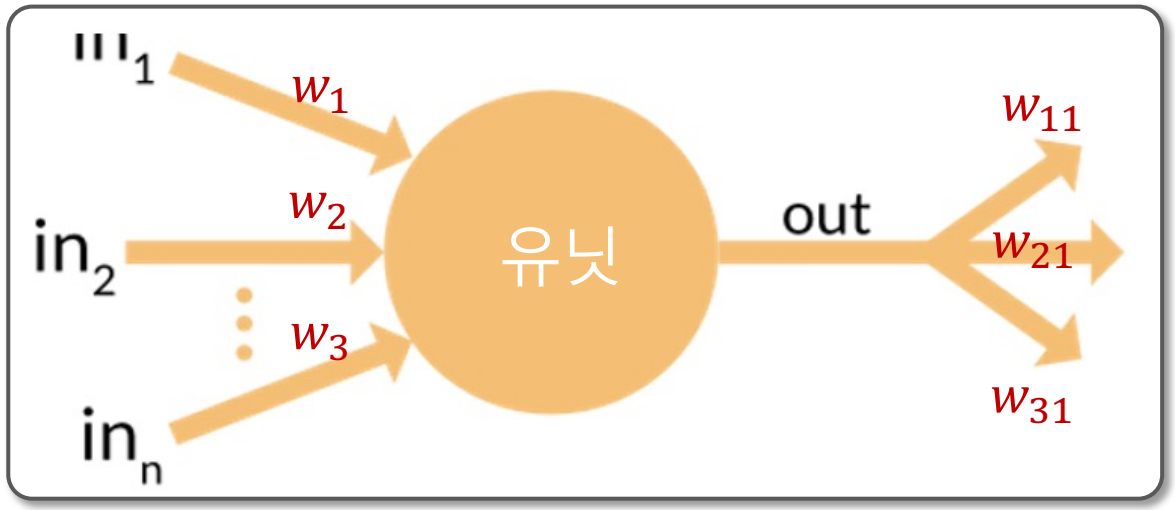
2. Nested Function
한 뉴런의 출력이 다음 뉴런의 입력이 됨

3. Activation Function
뉴런의 출력은 특정 활성을 통해서
다음 뉴런으로 전달되거나 전달되지 않음

1. 인공 신경망

인공 신경망(artificial neural networks)이란?

- 수학적 모델로서 인공신경망은 "유닛" 또는 "노드"라고 불리는 계산 단위들로 구성되며, 이들 노드는 가중치(weight)를 통해 서로 연결



유닛(혹은 노드)

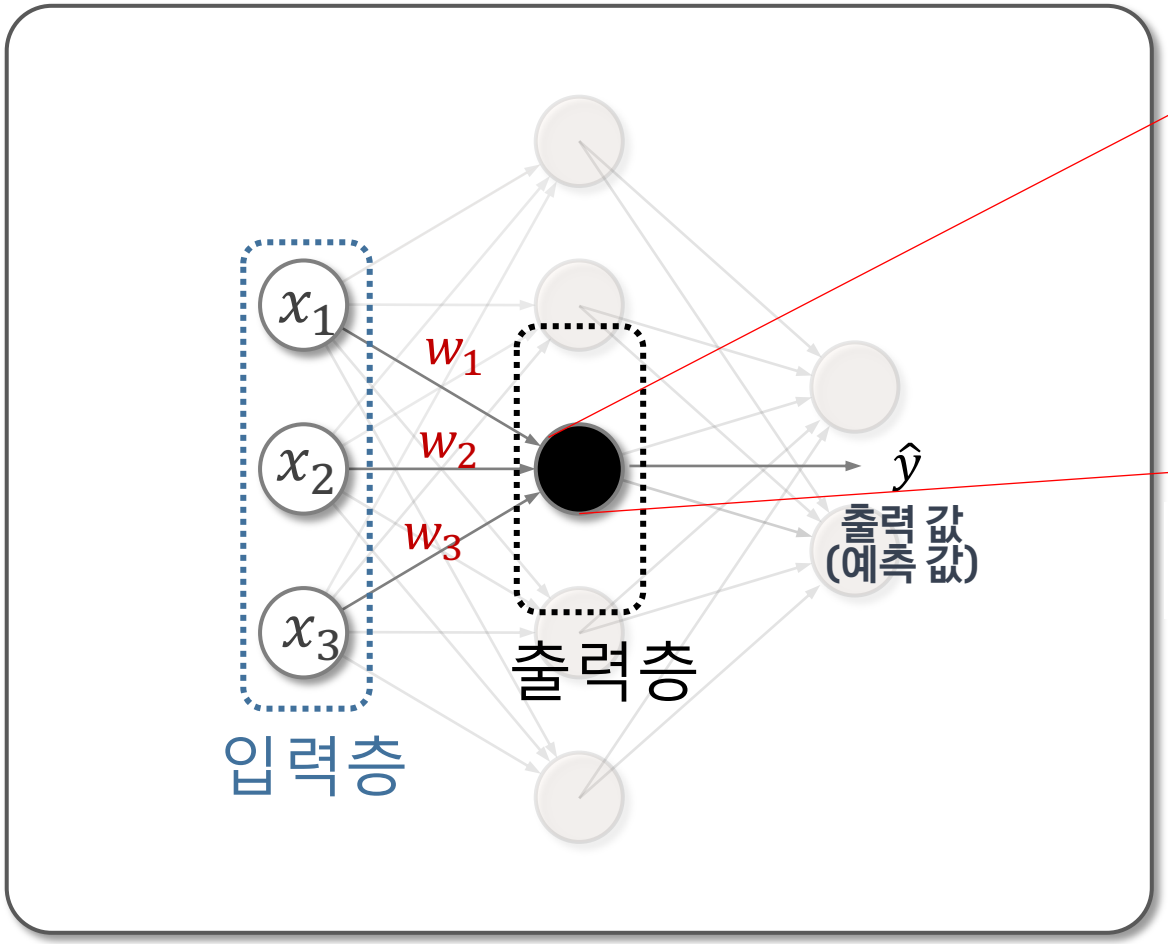
1. Function
유닛은 입력 신호를 받아서 출력 값을 생성

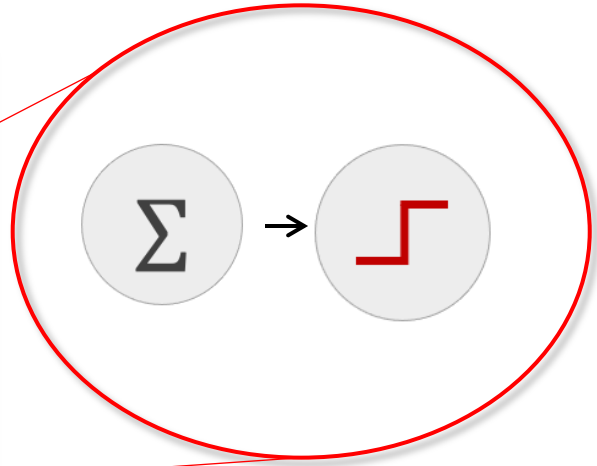
2. Nested Function
한 유닛의 출력이 다음 유닛의 입력이 됨

3. Activation Function
유닛의 출력은 특정 활성을 통해서
다음 유닛으로 전달되거나 전달되지 않음

1. 단층 퍼셉트론이란?

- 가중치(w)적용을 받는 층 1개로 구성된 가장 기본적인 피드포워드 신경망 구조





Σ 합성 함수 (Combination Function)

출력과 연관된 중요한 입력 값(x)에 대해서 좀 더 가중치(w)를 주고 모두 더해서 만든 값

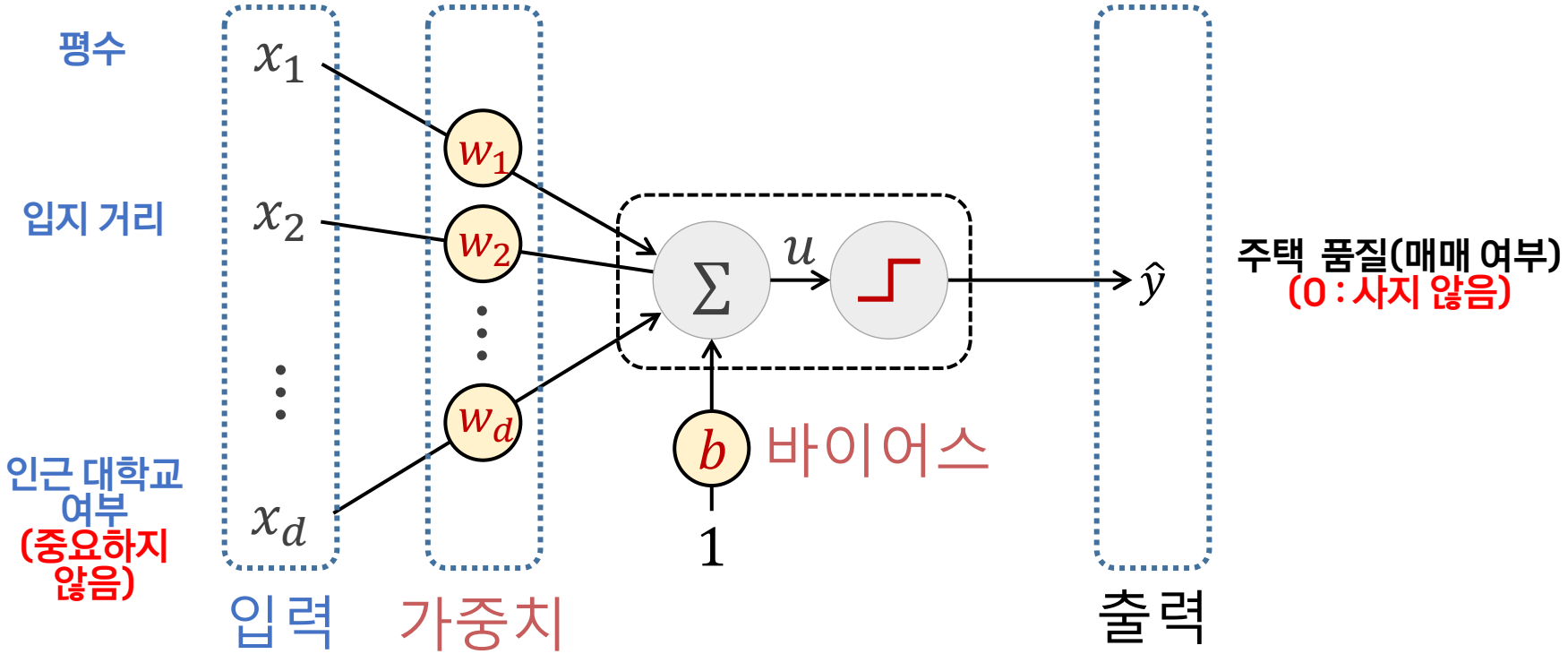
활성 함수 (Activation Function)

합성 함수로 계산된 결과값이 최종 임계치를 넘지 못하면 다음 노드로 전달하지 않고($\hat{y}$ == 0값 처리)

최종 임계치를 넘기면 다음 노드로 최종 결과($\hat{y}$)를 전달!!

1. 단층 퍼셉트론의 구조

- 가중치(w)적용을 받는 층 1개로 구성된 가장 기본적인 피드포워드 신경망 구조



주택 품질(매매 여부)
(0 : 사지 않음)

출력과 연관된 중요한 입력 값(x)에 대해서
좀 더 가중치(w)를 주고 모두 더해서 만든 값(u)이

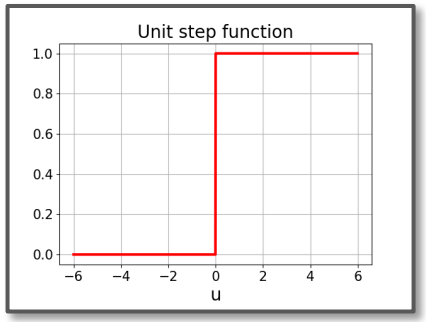
Σ 합성 함수 (Combination Function)

최종 임계치를 넘지 못하면 다음 노드로 전달하지 않고($\hat{y}$ == 0값 처리)
최종 임계치를 넘기면 다음 노드로 최종 결과($\hat{y}$)를 전달!!

활성 함수 (Activation Function)

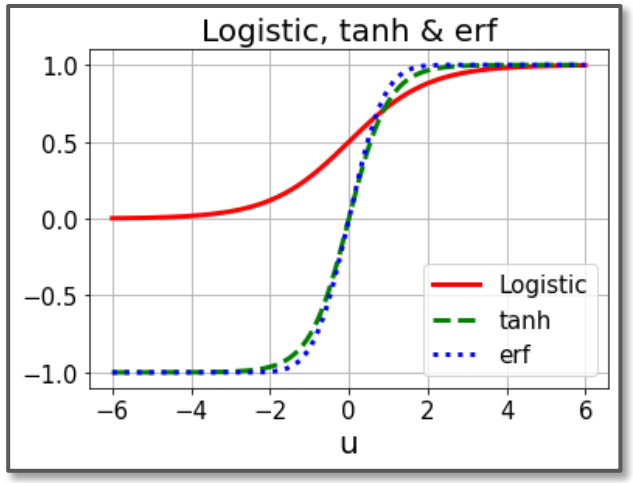
3. 활성(active)함수의 종류

계단 함수



$$\varphi_{step}(u) = \begin{cases} 1, & u \geq 0 \\ 0, & u < 0 \end{cases}$$

시그모이드 함수

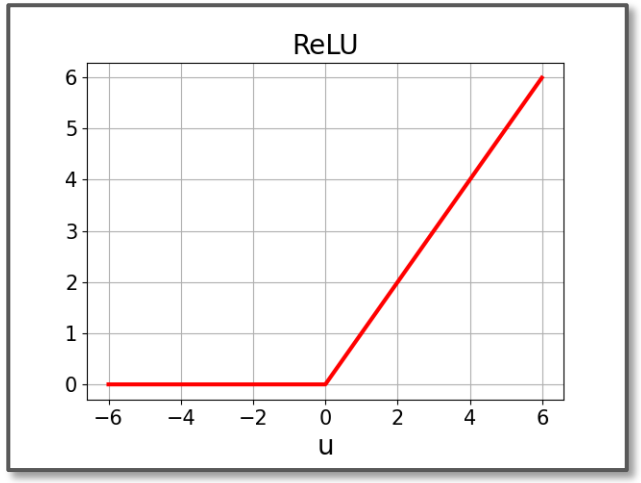


$$\varphi_{Logistic}(u) = \frac{1}{1+e^{-u}}$$

$$\varphi_{tanh}(u) = \frac{e^{u}-e^{-u}}{e^{u}+e^{-u}}$$

$$\varphi_{erf}(u) = \frac{2}{\sqrt{\pi}}\int_{0}^{u} e^{-t^2}dt$$

ReLU(Rectified Linear Unit)



$$\varphi_{ReLU}(u) = \begin{cases} 0, & u < 0 \\ u, & u \geq 0 \end{cases}$$

또는 $\varphi_{ReLU}(u) = \max(0, u)$

이진 클래스 분류(Binary Classification) 용도



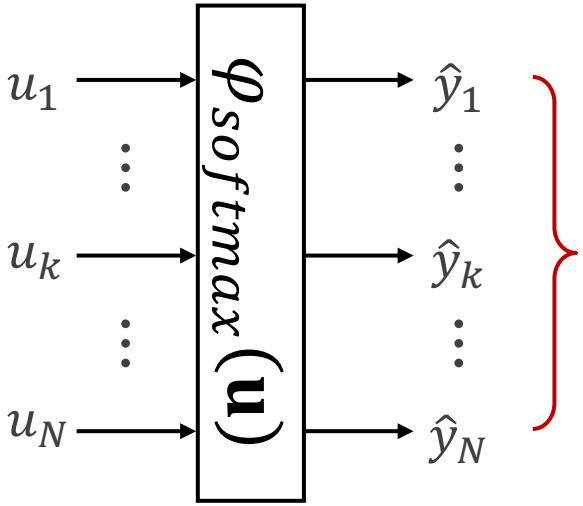
그럼 다중 분류는 어떻게 할까?

이진분류	다중 분류
	
5 or not?	1 or 2 or 3 or 4 or 5?

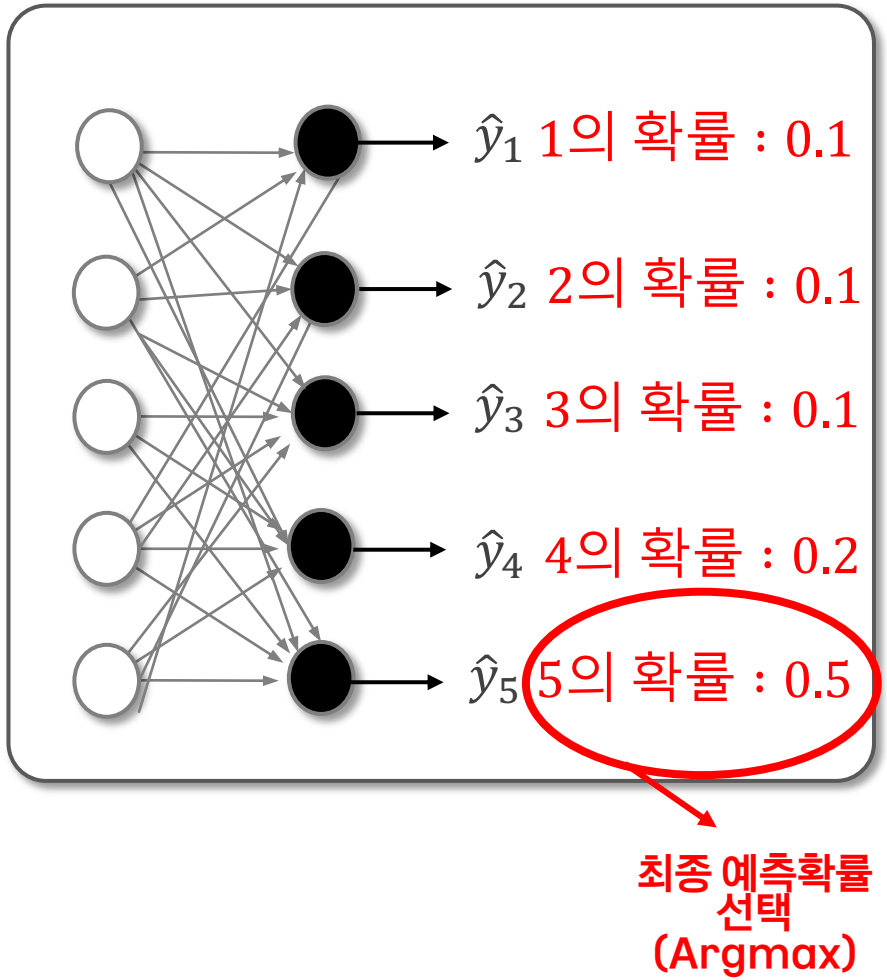
3. 활성(active)함수의 종류

소프트맥스(softmax)

- 다중 클래스 분류(Multiple Classification) 문제에서 출력층의 활성함수로 적합한 함수
- 출력층이 클래스 집합의 확률분포를 나타내는 값을 출력함



$$class = \underset{i}{\operatorname{argmax}}\, \hat{y}_i$$

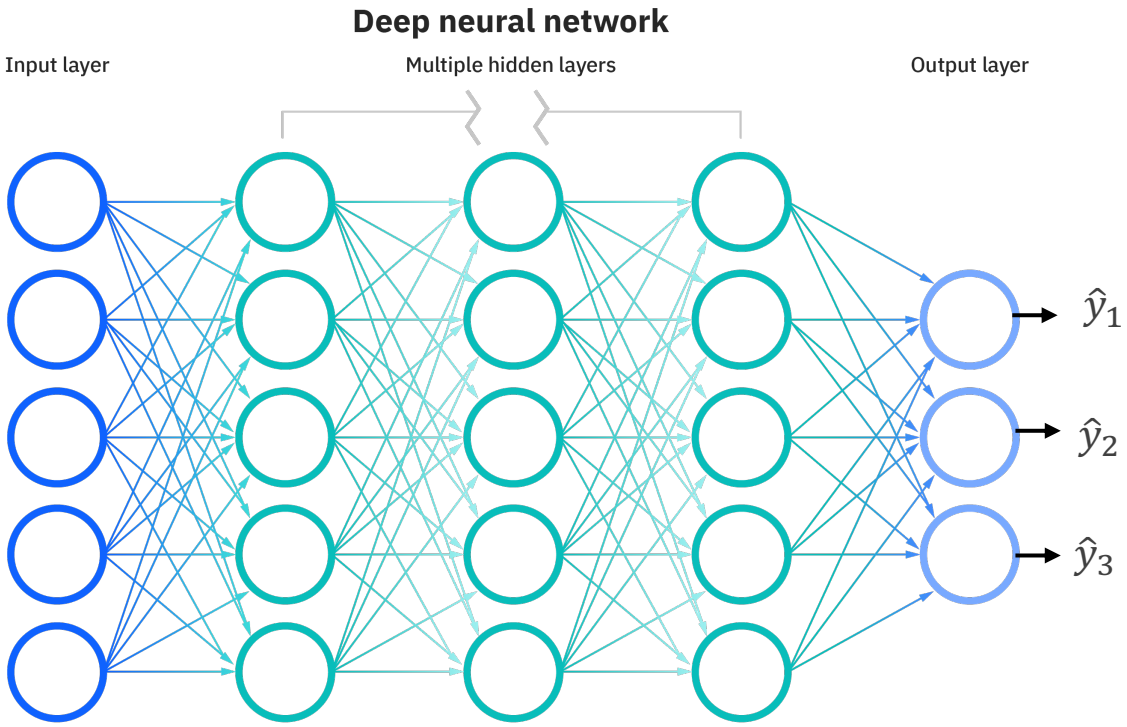


$$\varphi_{softmax}(\mathbf{u})_k = \frac{e^{u_k}}{\sum_{i=1}^{N} e^{u_i}}, \qquad k = 1,2,\cdots,N$$

2. 다층 퍼셉트론(Multi-Layer Perceptron)

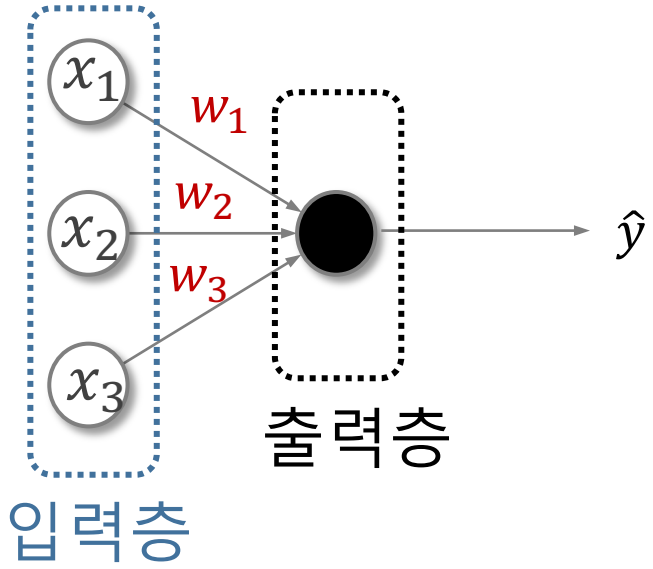
- 가중치(w)적용을 받는 층이 2개 이상 구성된 피드포워드 신경망 구조

다층 퍼셉트론



VS

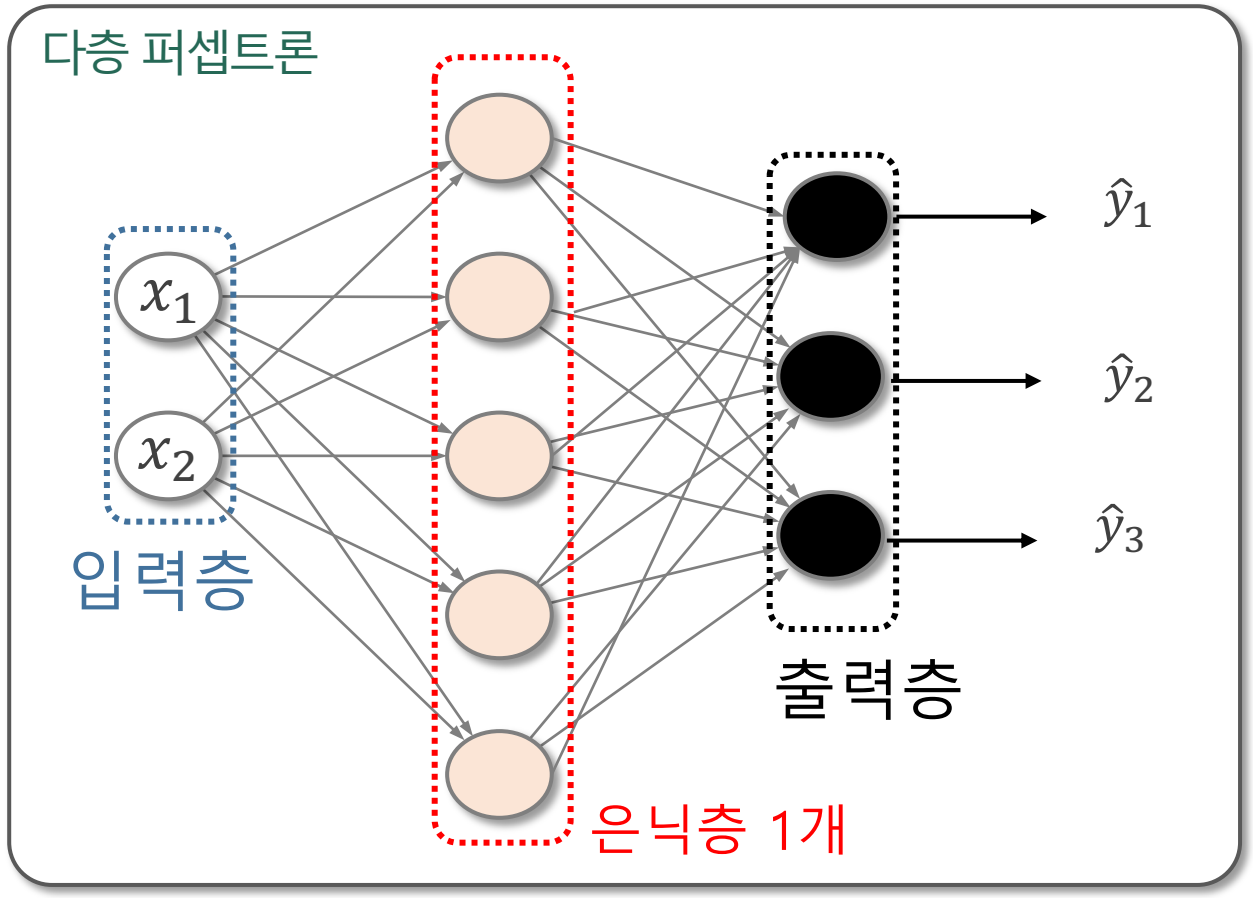
단층 퍼셉트론



은닉층 없음

2. 다층 퍼셉트론(Multi-Layer Perceptron)

- 잠깐 퀴즈!!!

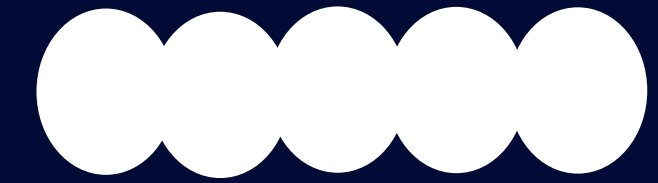
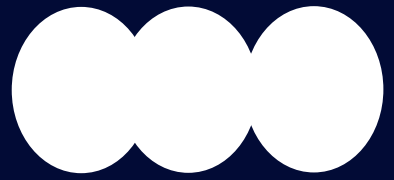


Q1. 전체 파라미터 (weight) 개수는 몇 개일까요?

Q2. 최적의 파라미터(weight)란?

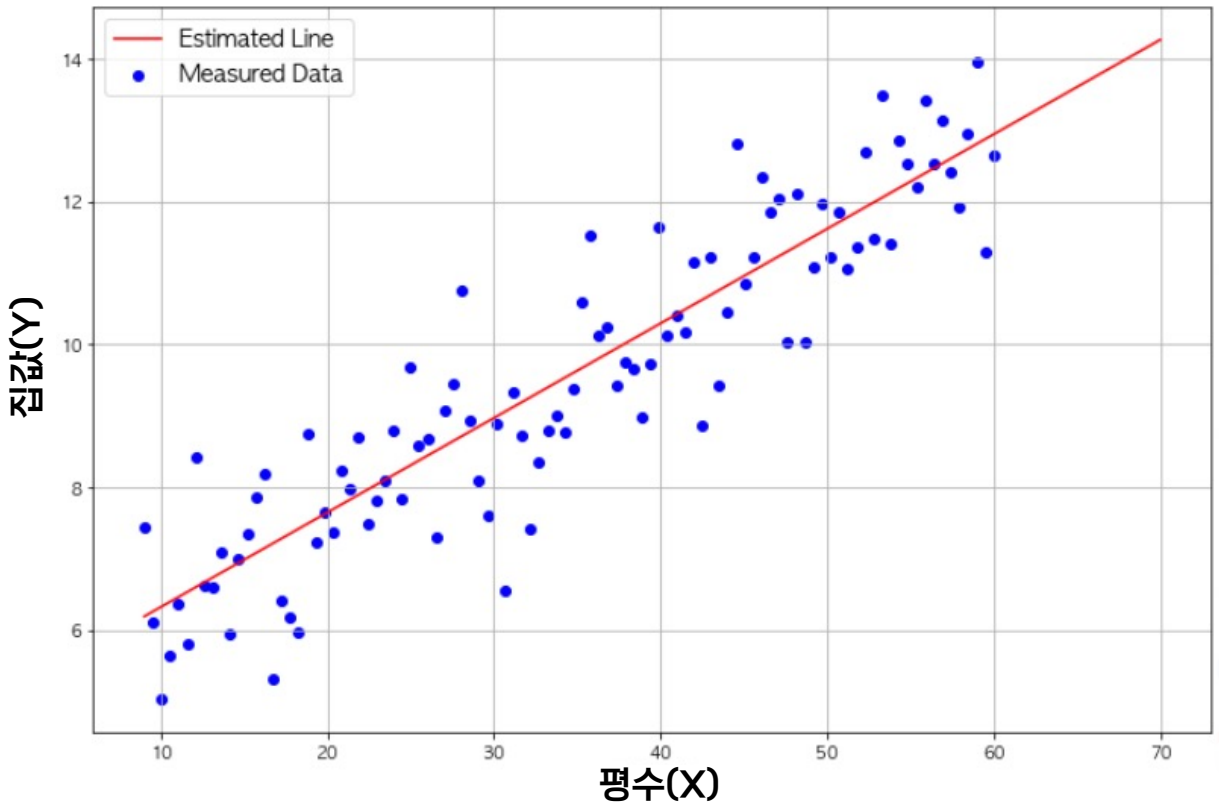
🔍 (예측이 정답에 가까워지는)

최적의 파라미터(weight)를 찾는 방법은?

1. 
2. 

Loss Function(or Cost Function)

손실 값(Loss or Cost or Error)은 예측 값($\hat{y}$) 과 실측 값(y)과 차이 값



예측 값($\hat{y}$) = wX + b



Mean Squared Error $= \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$

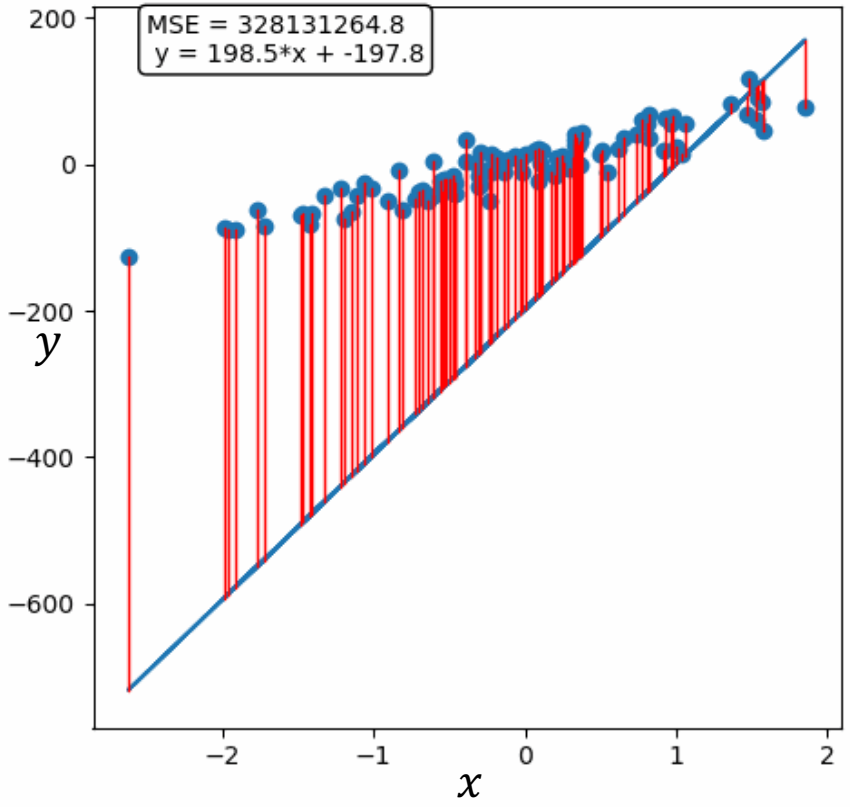
$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - (wx_i + b))^2$

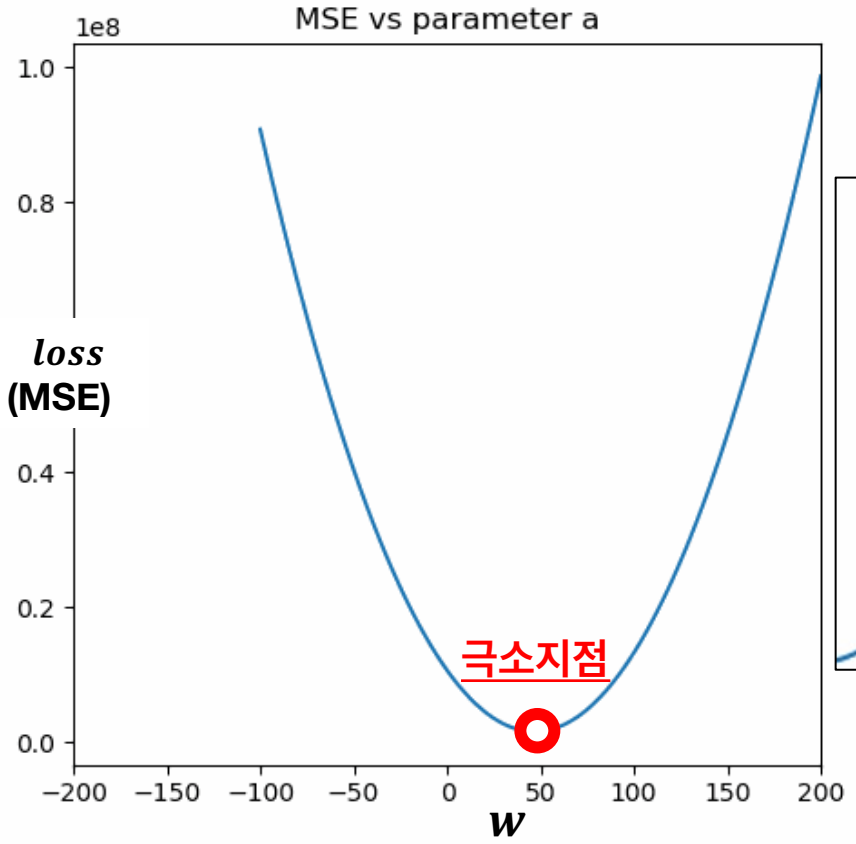
MSE(혹은 loss) = w^2X +...

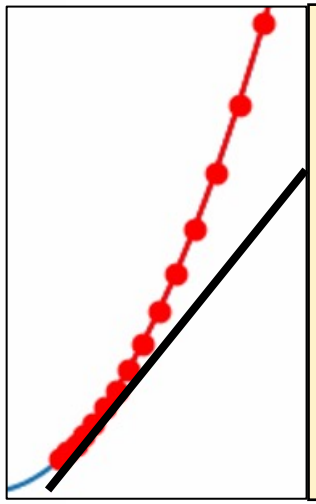
w의 2차 함수 형태(y축은 loss)

경사 하강법(Gradient Decent)

손실 값(loss)에 w가 미치는 영향(틀린 정도)를 알고 적절하게 w를 업데이트 시킨다면?







어떤 방향(기울기)으로 내려갈까?

기울기 (gradient)

$$\frac{\partial loss}{\partial w}$$

예측 값($\hat{y}$) = wx + b

loss = $xw^2 + \cdots$

경사 하강법(Gradient Decent)

손실 값(loss)에 [illegible]를 알고 적절하게 w를 업데이트 시킨다면?

여기선 잠깐 🛑,

간단한 수학 원리 복습시간! 📚

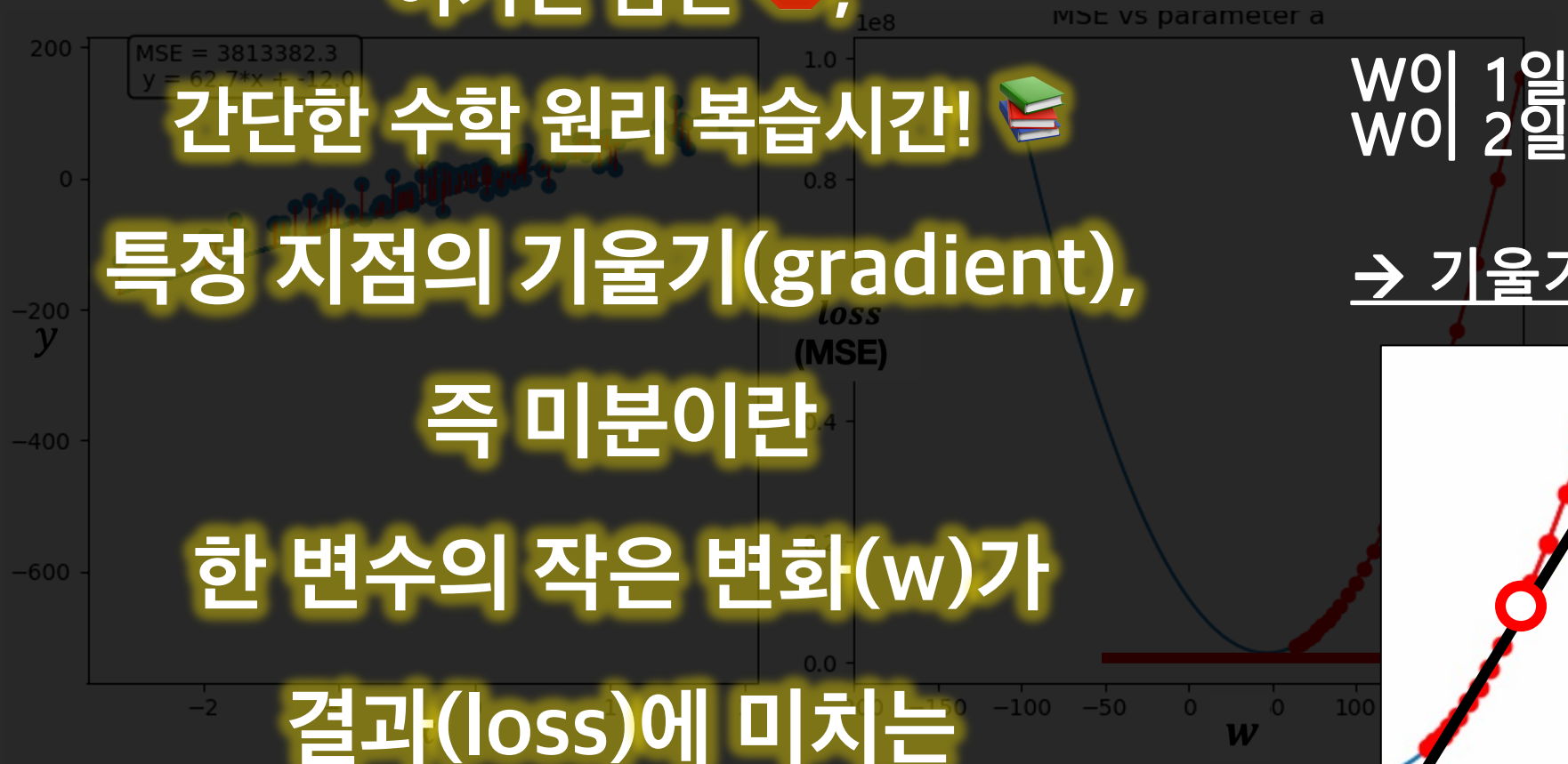
특정 지점의 기울기(gradient),

즉 미분이란

한 변수의 작은 변화(w)가

결과(loss)에 미치는

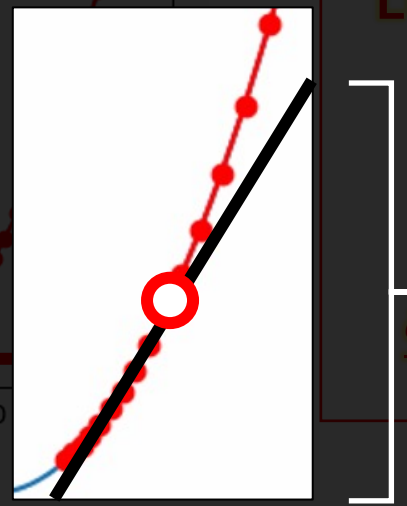
순간 변화량을 의미한다



$loss = 2w$

W이 1일때, loss는 +2
W이 2일때, loss는 +4

→ 기울기가 2($\frac{dloss}{dw} \approx \frac{2}{1} = \frac{4}{2}$)



W가 +1만큼 커질 때

Loss가 +2만큼 커짐

$loss = xw^2 + \cdots$

[illegible] 정도가
가장 작은 경우
(최적의 w)

Loss == 0인 경우의 w

or

현 지점의 기울기
(gradient)

$\frac{\partial loss}{\partial w}$ == 0 경우의 w

경사 하강법(Gradient Decent)

손실 값(loss)에 [illegible]를 알고 적절하게 w를 업데이트 시킨다면?

$loss = 2w + b$

여기선 잠깐 🛑,

간단한 수학 원리 복습시간! 📚

(b 와 상관없이) w가 1일 때, loss는 +3
(b 와 상관없이) w가 2일 때, loss는 +6

편 미분(∂)이란

다 변수 상황에서

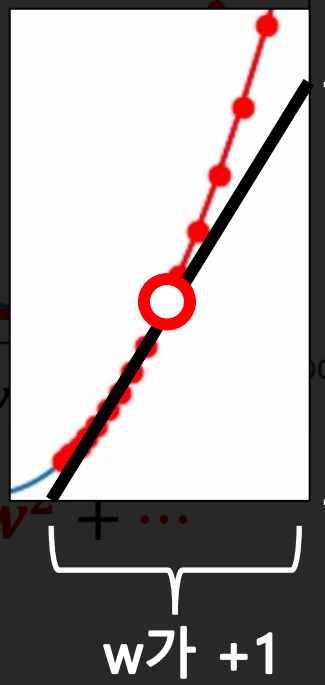
한 변수의 작은 변화만(타 변수 무시)

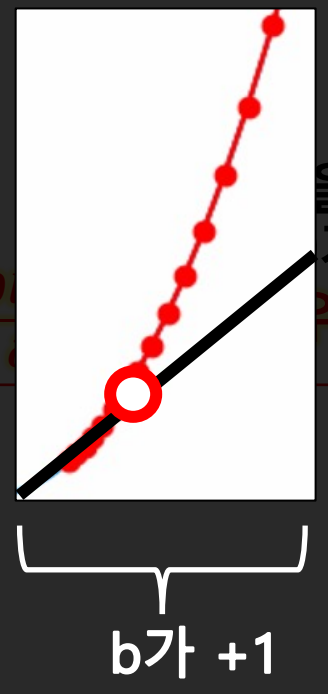
결과에 미치는

편중된 순간 변화량을 의미한다

$\rightarrow \frac{\partial loss}{\partial w} = 2 \approx \frac{2}{1} = \frac{4}{2}$

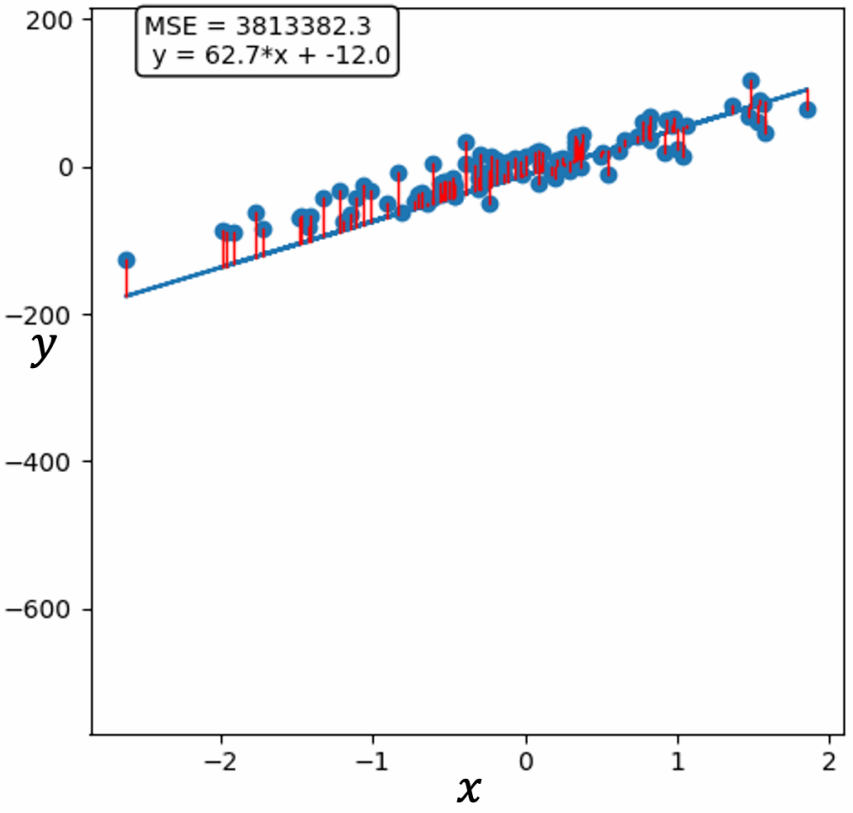
$\frac{\partial loss}{\partial b} = 1 \approx \frac{1}{1}$

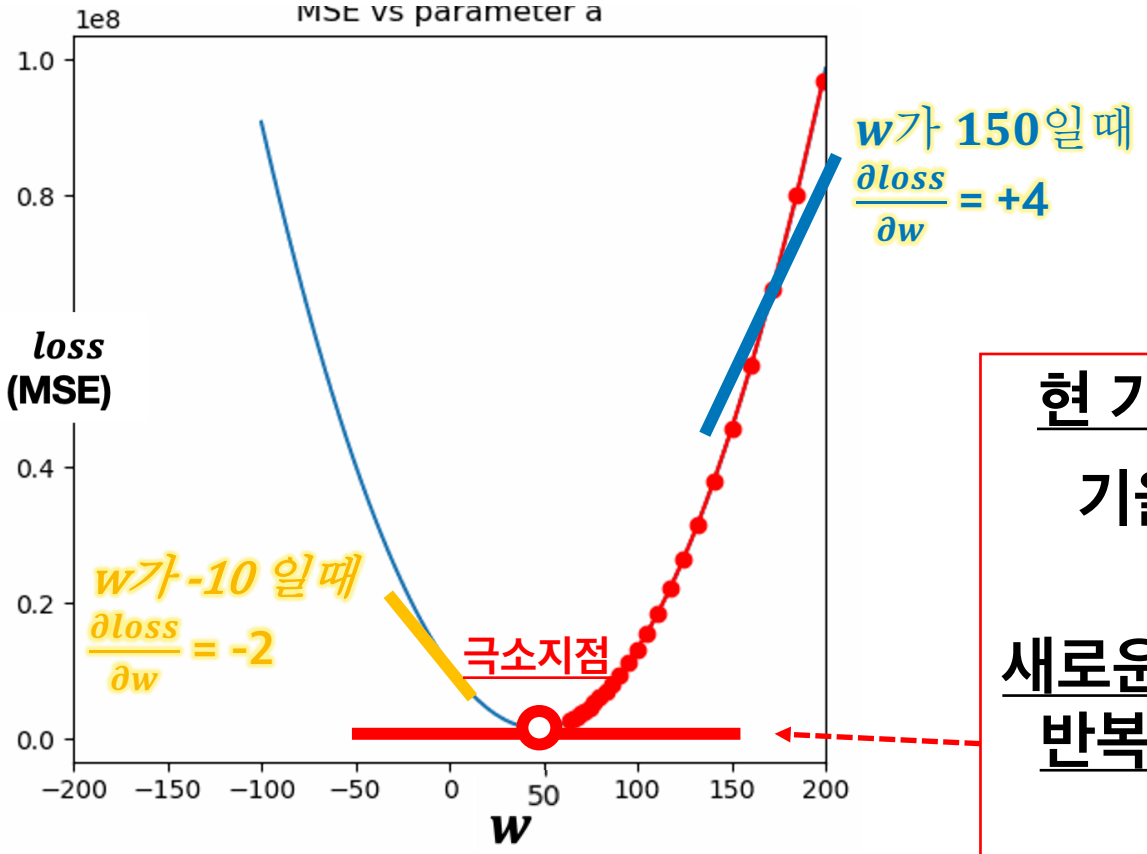




경사 하강법(Gradient Decent)

손실 값(loss)에 **w가 미치는 영향(틀린 정도)**를 알고 **적절하게 w를 업데이트** 시킨다면?





예측 값($\hat{y}$) = $wx + b$

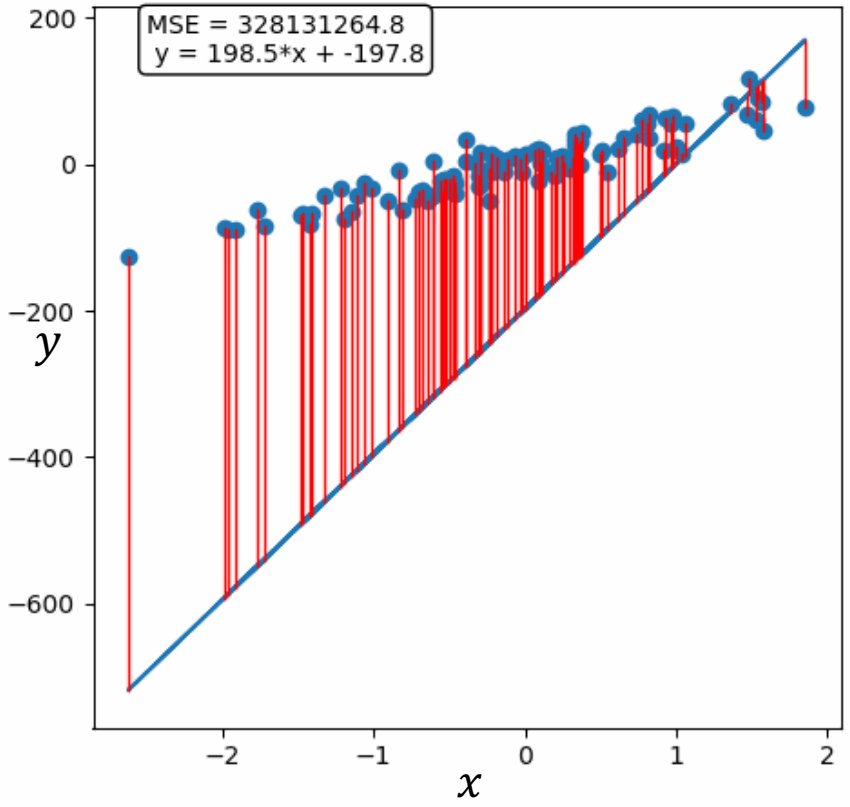
loss = $xw^2 + \cdots$

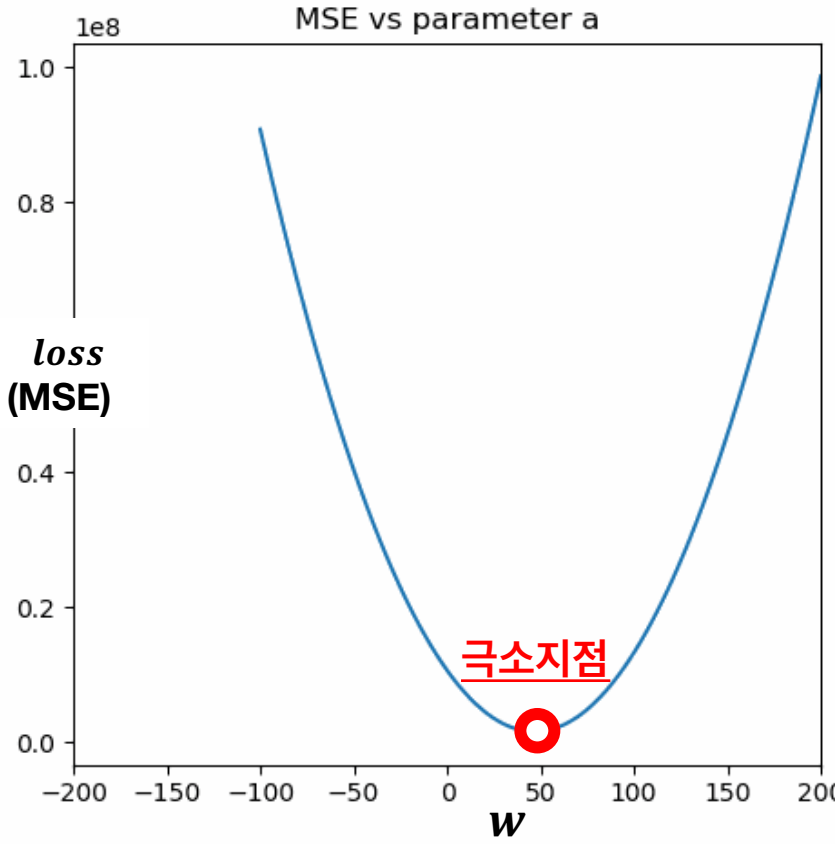
현 가중치(w_{old})에서
기울기($\frac{\partial loss}{\partial w}$)만큼
뺀(-)
새로운 가중치(w_{new})로
반복적 업데이트하며

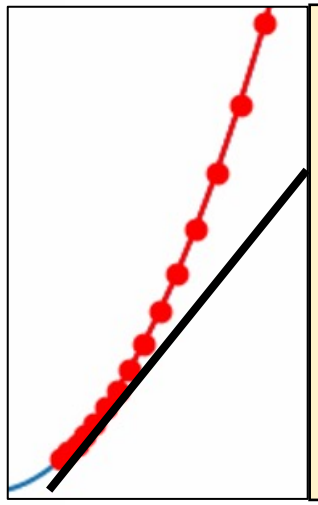
극소 지점을 향한다
$\frac{\partial loss}{\partial w} \approx 0$ 일 때의
$w \approx 50$

경사 하강법(Gradient Decent)

손실 값(loss)에 w가 미치는 영향(틀린 정도)를 알고 적절하게 w를 업데이트 시킨다면?

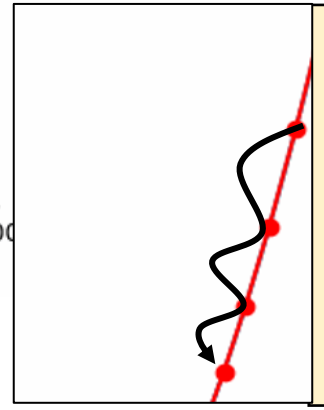






1.어떤 방향(기울기)으로 내려갈까?

기울기
(gradient)
$\frac{\partial loss}{\partial w}$



2.어떤 보폭 크기로 내려갈까?

보폭 크기
(learning rate)
η

예측 값($\hat{y}$) = wx + b

loss = $xw^2 + \cdots$

경사 하강법(Gradient Decent)

기울기(gradient)가 0이 될 때까지 weight 업데이트를 통해 loss의 최소지점(극소점)을 찾는 최적화 기법

$$w_{new} = w_{old} - \eta\left(\frac{\partial loss}{\partial w}\right)$$

weight의 업데이트 = Loss 낮추는 방향 *(decent)* X 현 지점의 기울기 (gradient) X 보폭 크기 (learning rate)

$$-\eta\left(\frac{\partial loss}{\partial w}\right) \quad - \quad \frac{\partial loss}{\partial w} \quad \eta$$

최적의 weight(손실 최소화)

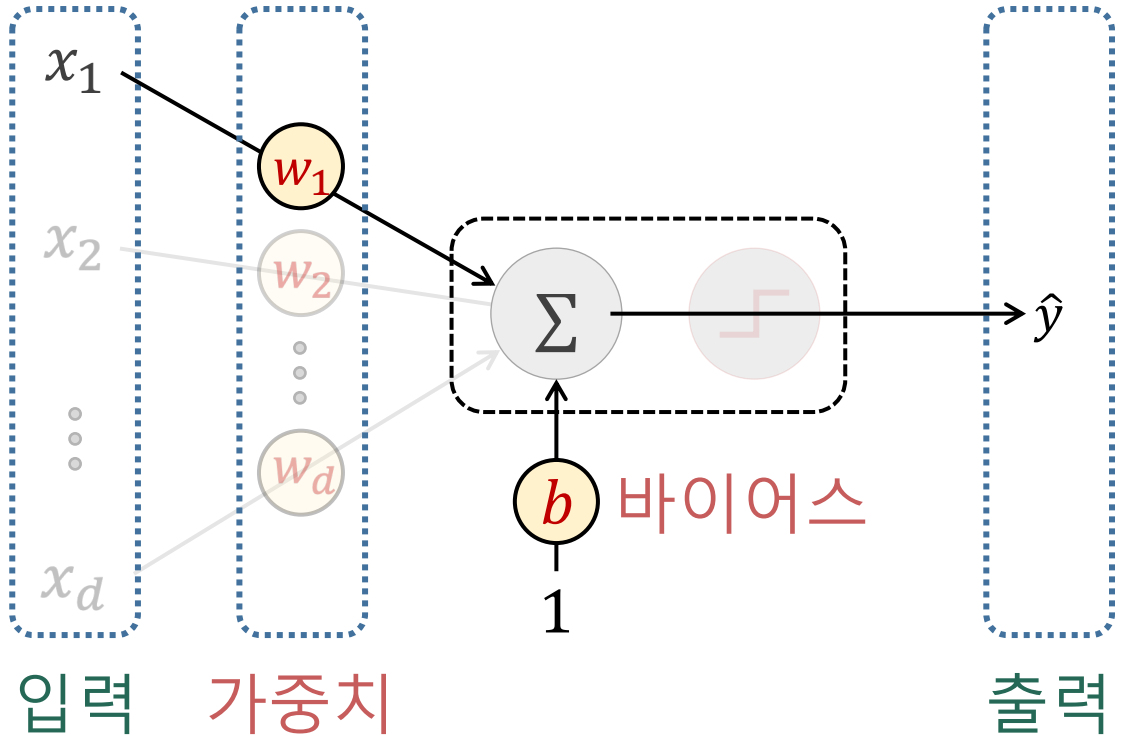
계산 방법인 경사 하강법을

터득(?)했으니

이제 다 해결된 것일까요?

Model Complexity(Easy)

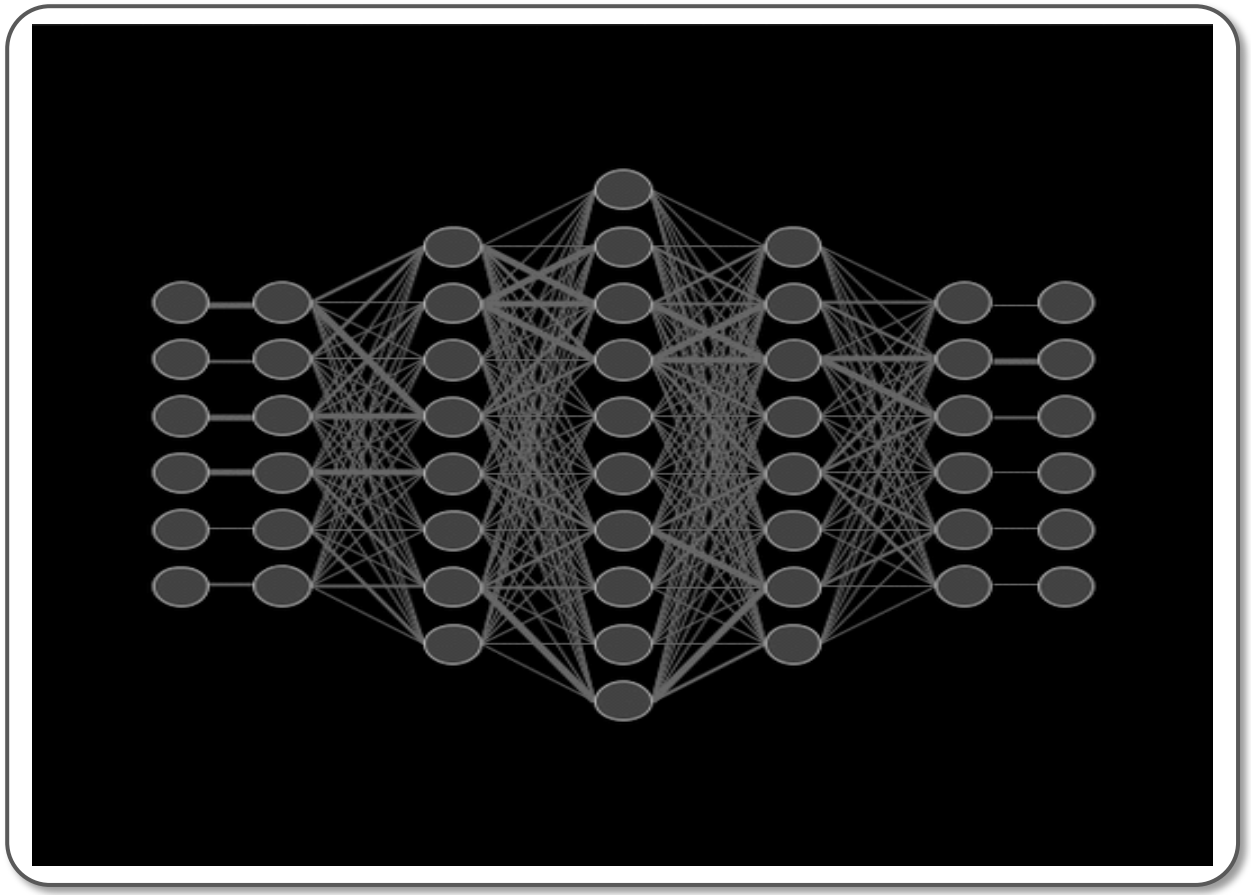
이전 예시 함수는 단순해서(Not a Nested Function) 경사 하강 법 적용이 쉬운 편



예측 값($\hat{y}$) $= w_1 x + b$

Model Complexity(Hell)

But 딥러닝 모델 함수는 많이 복잡하다(Deeply Nested Function)



- Σ 합성 함수 (Combination Function)
- 활성 함수 (Activation Function)
- Σ 합성 함수 (Combination Function)
- 활성 함수 (Activation Function)
- Σ 합성 함수 (Combination Function)
- 활성 함수 (Activation Function)
- ⋮

예측 값($\hat{y}$) =

$$\varphi_{active}(w_2\varphi_{active}\ (w_1 \mathrm{x} + b_1) + b_2)\cdots$$

🤔

복잡한 딥러닝 모델의

파라미터($w_1, w_2, \cdots, w_n$)에 대해서도

각각 최적의 손실 최소화를

하고 싶다..



파라미터($w_1, w_2, \cdots, w_n$)에 대해서

손실($loss$)에 미친

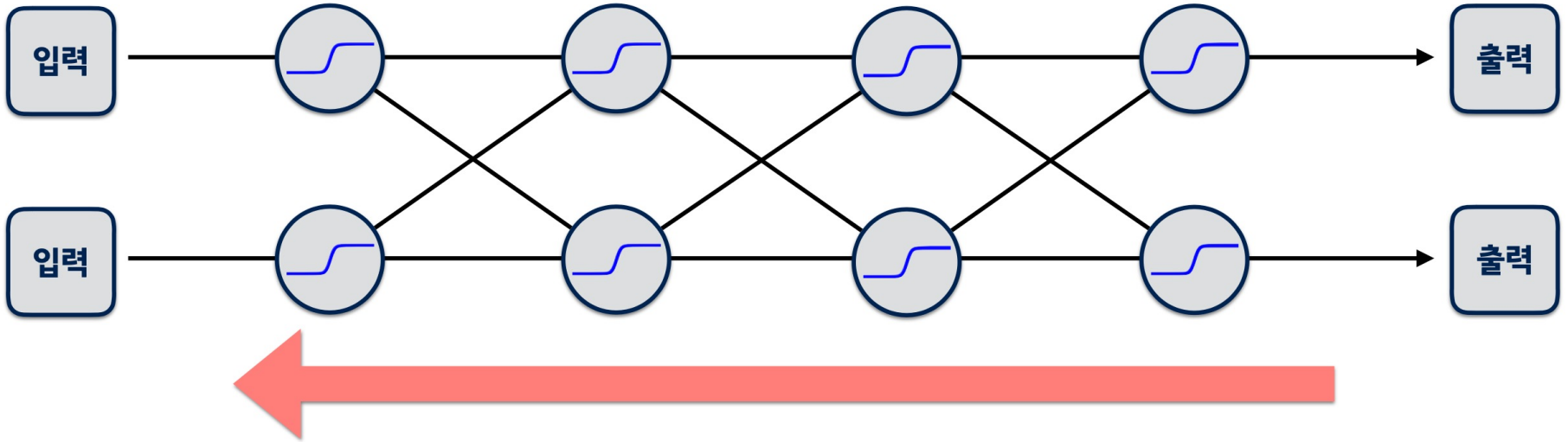
틀린 정도(Gradient)를

각각 계산할 수 있다면??

역전파(Back Propagation)

출력(예측) 기준, 맨 마지막 순서부터 역방향으로 각각의 weight에 대한 미분 값(기울기)을 계산하는 기법

현재 내가 틀린정도를 '미분(기울기)' 한 거



딥러닝 출석수업

02

딥러닝의 기본 구현

1. 주요 딥러닝 프레임워크 소개
2. 텐서
3. Keras를 이용한 모델의 구현



1. 주요 딥러닝 프레임워크 소개

딥러닝 프레임워크란?



- 딥러닝 알고리즘을 쉽고 효과적으로 구현하기 위한 소프트웨어 라이브러리
- 빠르게 딥러닝 모델을 개발, 테스트, 튜닝할 수 있도록 다양한 도구와 기능을 제공
- 딥러닝 프레임워크 대부분 파이썬 언어 호환이 잘 됨

1. 주요 딥러닝 프레임워크 소개

주요 딥러닝 프레임워크 소개

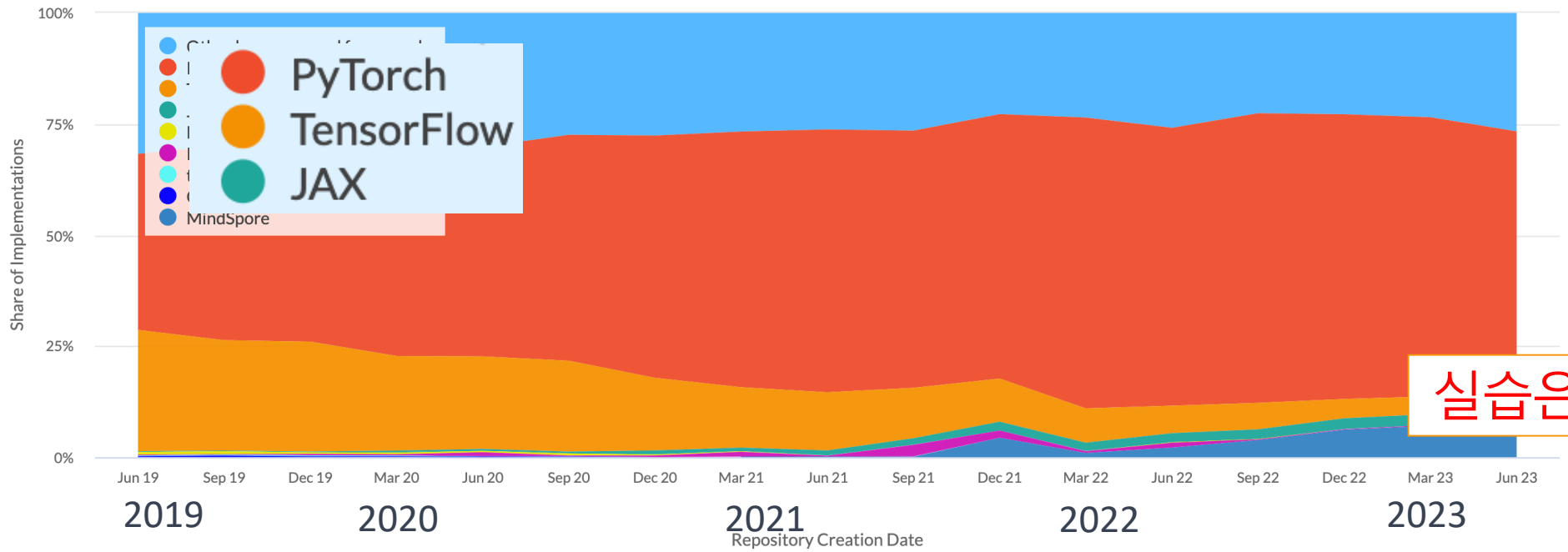
Trends

Quarter | 2019-05-29 to 2023-06-30

Frameworks

Paper Implementations grouped by framework

- 현재 주요 딥러닝 프레임워크 : Pytorch, TensorFlow(+Keras), Jax등
- 사용성 우수한 순 : TensorFlow(**Keras**) >= Pytorch >= Jax



실습은 only Keras로 진행

딥러닝 출석수업

02

딥러닝의 기본 구현

1. 주요 딥러닝 프레임워크 소개
2. 텐서
3. Keras를 이용한 모델의 구현



1. 텐서의 개념

- 텐서(tensor)
  - 동일 자료형의 데이터(대부분 부동 소수점)를 저장하는 다차원 배열
  - numpy의 배열과 유사한 형태로 데이터를 저장하는 객체

rank-0D 텐서
(스칼라)

5

shape: []
or
shape=()

rank-1D 텐서
(벡터)

5	-2	3

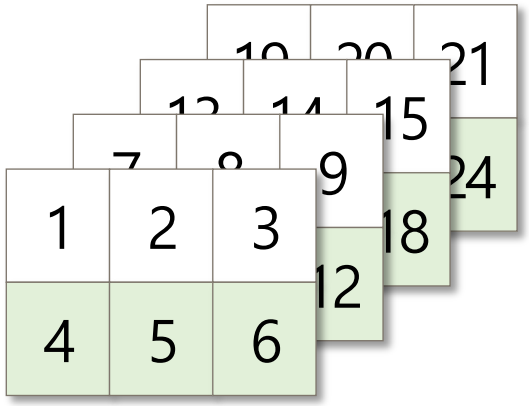
shape: [3]
or
shape=(3,)

rank-2D 텐서
(행렬)

1	2	3
4	5	6

shape: [2, 3]
or
shape=(2, 3)

rank-3D 텐서
(다차원 배열)



shape: [4, 2, 3]

3D → 2D

1	2	3
4	5	6
7	8	9
10	11	12
13	14	15
16	17	18
19	20	21
22	23	24

shape: [8, 3]

1. 텐서의 개념

주로 사용되는 텐서 예시

- 일반 표 데이터(2D, 1D Tensor)

2D 입력 feature 데이터 (9x3)

평수(X1)	건축년도(X2)	송전탑 거리 (X3, m)
25	1985	300
20	1990	100
28	1995	120
35	2000	150
30	2005	200
40	2010	50
45	2015	75
38	2020	90
34	2021	150

9

3

shape: [9, 3]

1D 정답 데이터

집값 (Y, 만원)
4,500
5,000
5,500
6,500
7,000
9,000
10,000
8,500
9,000

9

or

9

집값 (Y, 만원)
4,500
5,000
5,500
6,500
7,000
9,000
10,000
8,500
9,000

shape: [9]

1. 텐서의 개념

주로 사용되는 텐서 예시

- 일반 표 데이터(2D → 3D Tensor)

2D 입력 feature 데이터 (9x3)

평수(X1)	건축년도(X2)	송전탑 거리 (X3, m)
25	1985	300
20	1990	100
28	1995	120
35	2000	150
30	2005	200
40	2010	50
45	2015	75
38	2020	90
34	2021	150

9

3

shape: [9, 3]

2D → 3D



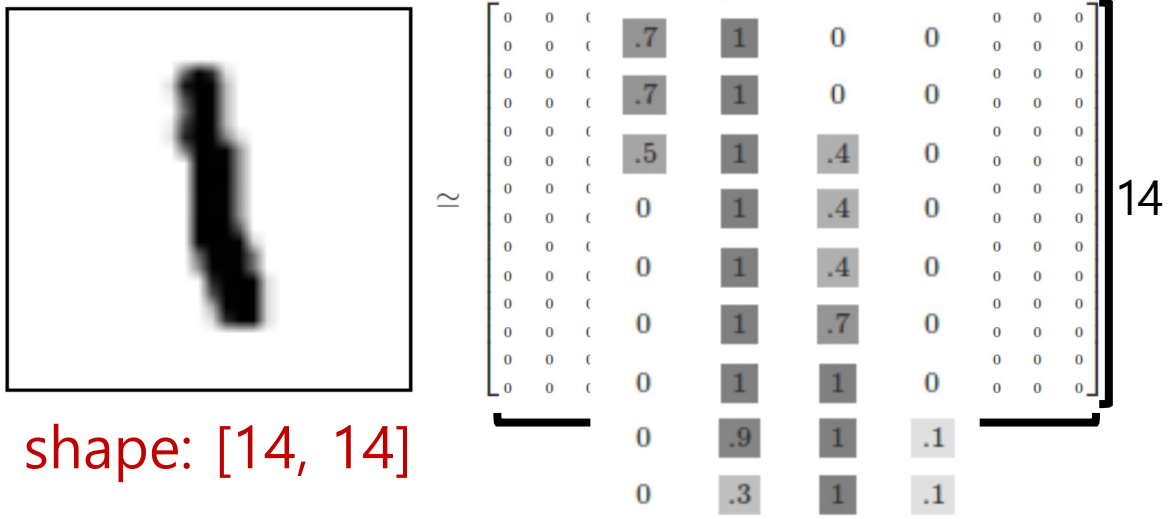
shape: [**3**, 3, 3]

1. 텐서의 개념

주로 사용되는 텐서 예시

- 이미지 데이터(2D, 3D Tensor)

2D 이미지 데이터(14x14)

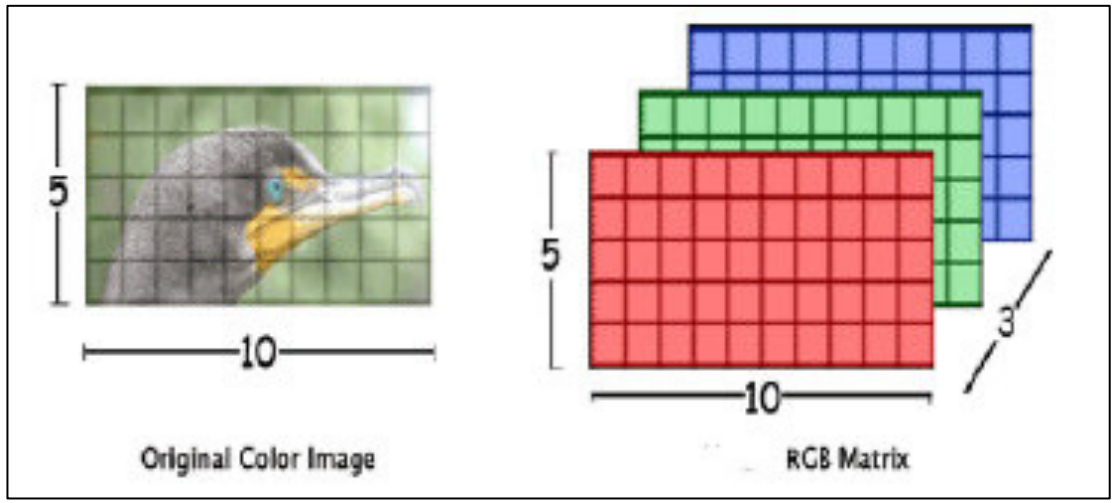


shape: [14, 14]

축소 이미지(14x14) 픽셀 개수 : 총196개

*실제 이미지(28x28) 픽셀 개수 : 총784개

3D 이미지 데이터(3x5x10) or 5x10x3



shape: [3, 5, 10]

channel, height, width

(CHW 형태)

이미지 차원 구성은 달라질 수 있음

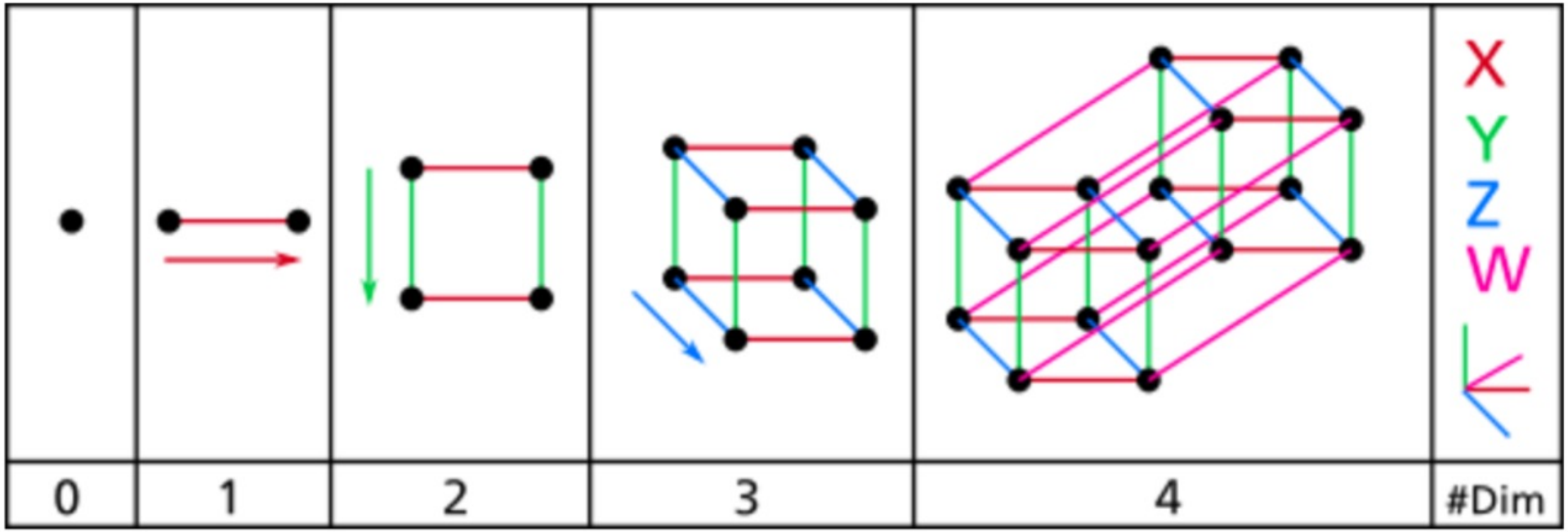
shape: [5, 10, 3]

height, width, **channel**

(HWC 형태)

1. 텐서의 개념

- **입력 Feature가 더 늘어나는 만큼 4D, 5D, ... N차원 확장**



일반적으로 데이터 시각화(그래프)로 가능한 차원수 범위

딥러닝 출석수업

02

딥러닝의 기본 구현

1. 주요 딥러닝 프레임워크 소개
2. 텐서
3. Keras를 이용한 모델의 구현



1. Keras의 개요



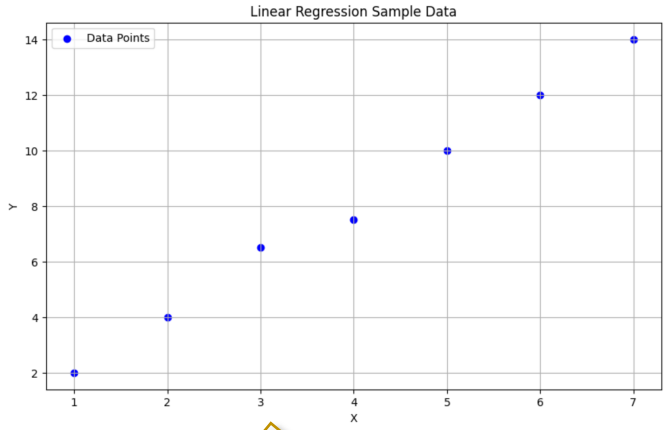
- Keras란?
  - 신경망 구현을 위한 고수준 오픈소스 라이브러리
  - 2.4 버전부터는 텐서플로만 지원
  - 텐서플로의 `tf.keras` 모듈에 제공됨
    - 여러 가지 신경망 층, 활성함수, 손실함수, 최적화기 등을 제공
- Keras를 이용하여 신경망 모델을 구성하는 방법
  - `tf.keras.Sequential` 클래스를 이용한 순차적으로 연결된 모델 구성
  - 함수형(functional) API를 이용한 모델 구성
  - 서브클래싱(subclassing) API를 이용한 모델 구성

→ **개발자가 아이디어에서 결과까지 적은 코드 라인으로 구현가능**

→ **활용 가능한 딥러닝 모델 : SLP, MLP, CNN, RNN, GAN, Transformer, etc**

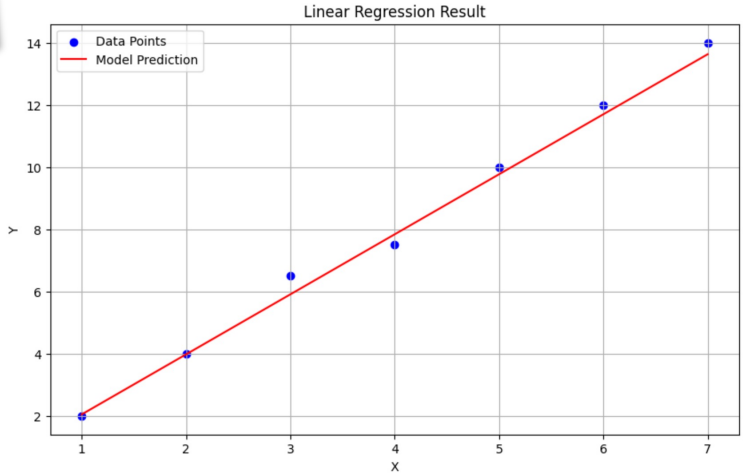
2. 실습 : 텐서플로를 이용한 선형 회귀

- hello world 의 keras 버전 :선형회귀(linear regression)



데이터 정의

```
[7]  # 데이터 정의
     x = [1, 2, 3, 4, 5, 6, 7] #shape=(7,)
     y = [2, 4, 6.5, 7.5, 10, 12, 14] #실제 정답, #shape=(7,)
```



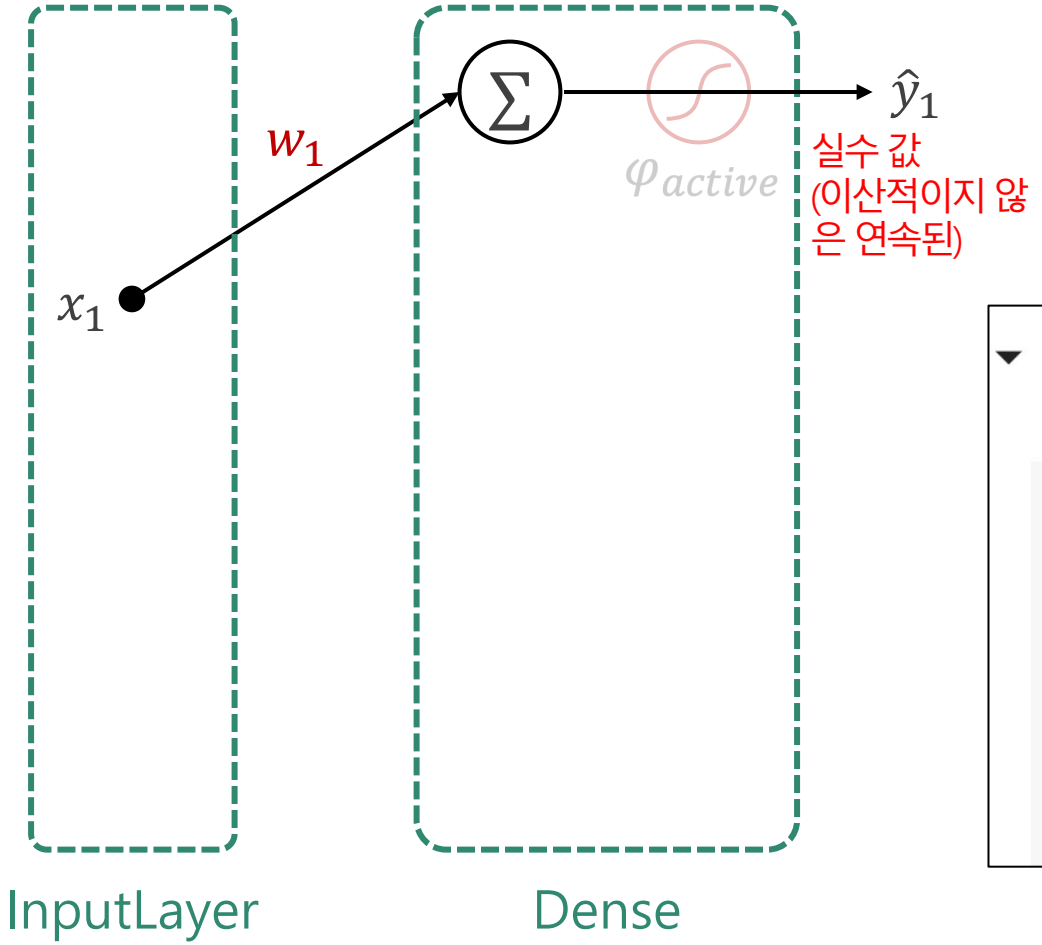
예측

```
[13] model.predict([7.1]) #shape=(1,)
     #model.predict(X) #shape=(7,)

     1/1 [==============================] - 0s 68ms/step
     array([[13.021487]], dtype=float32)
```

2. 실습 : 텐서플로를 이용한 선형 회귀

- 구현할 선형 회귀 : 모델 생성시, input_shape은 행 1개 단위(테스트 데이터 1개) 적용



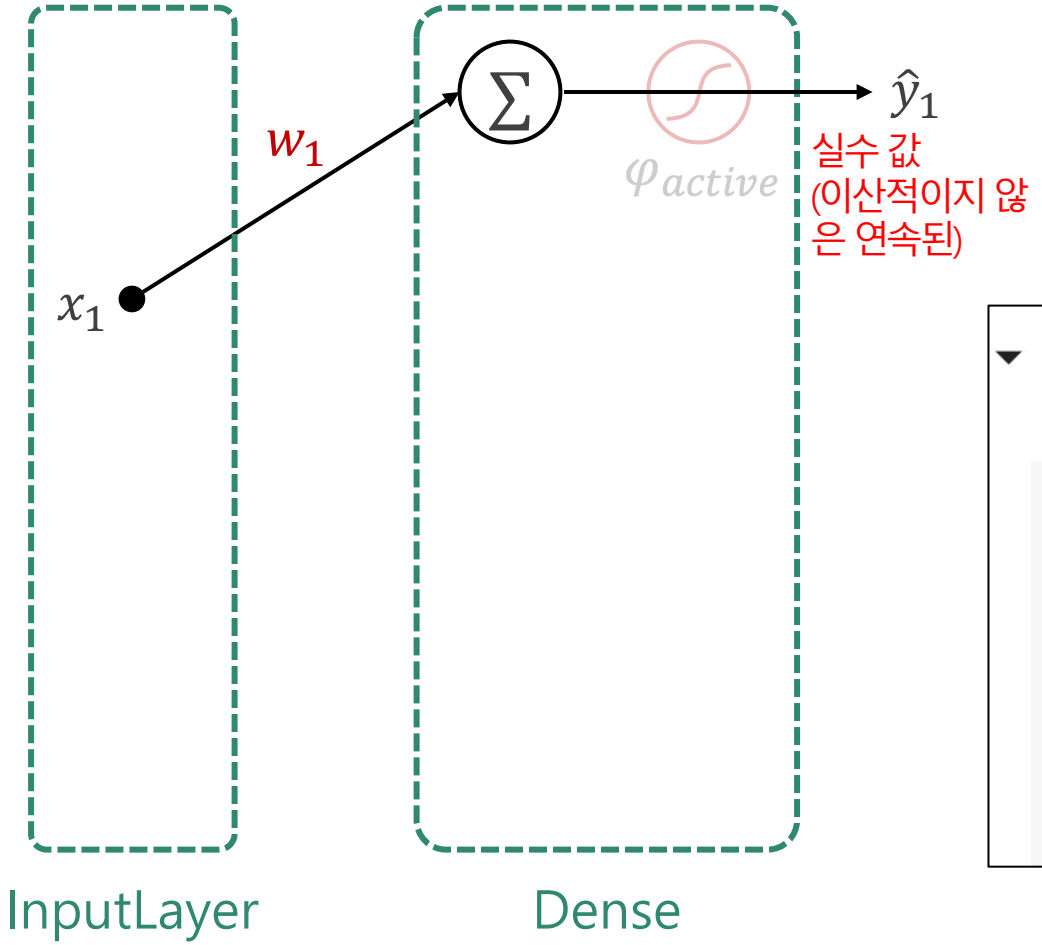
모델 구축 및 학습

```
[ ]  # 간단한 모델 구축(linear_regression) -> y_hat = wX + b
     model = tf.keras.Sequential([
         tf.keras.layers.Dense(units=1, input_shape=(1,))
     ])
     model.compile(optimizer='sgd', loss='mean_squared_error')

     # 모델 훈련(train)
     history = model.fit(X, y, epochs=5)
```

2. 실습 : 텐서플로를 이용한 선형 회귀

- 구현할 선형 회귀 : 손실(loss) 함수의 경우, MSE(Mean Squared Error)기법 활용



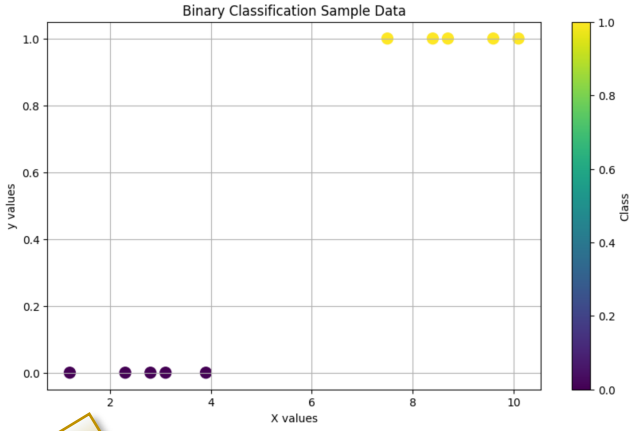
모델 구축 및 학습

```
[ ]  # 간단한 모델 구축(linear_regression) -> y_hat = wX + b
     model = tf.keras.Sequential([
         tf.keras.layers.Dense(units=1, input_shape=(1,))
     ])
     model.compile(optimizer='sgd', loss='mean_squared_error')

     # 모델 훈련(train)
     history = model.fit(X, y, epochs=5)
```

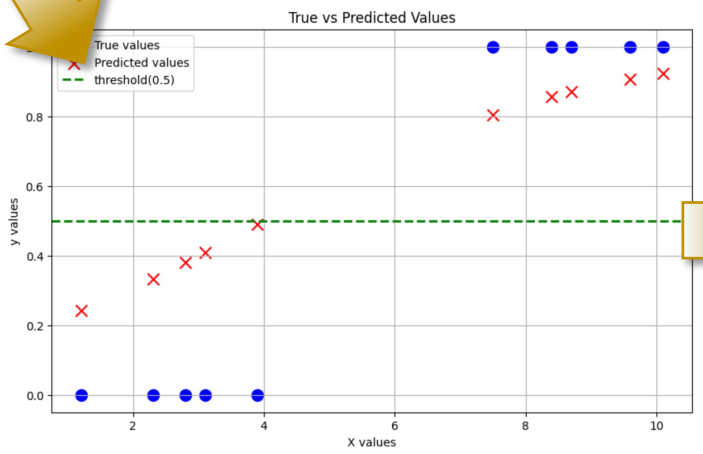
2. 실습 : 텐서플로를 이용한 단층 퍼셉트론

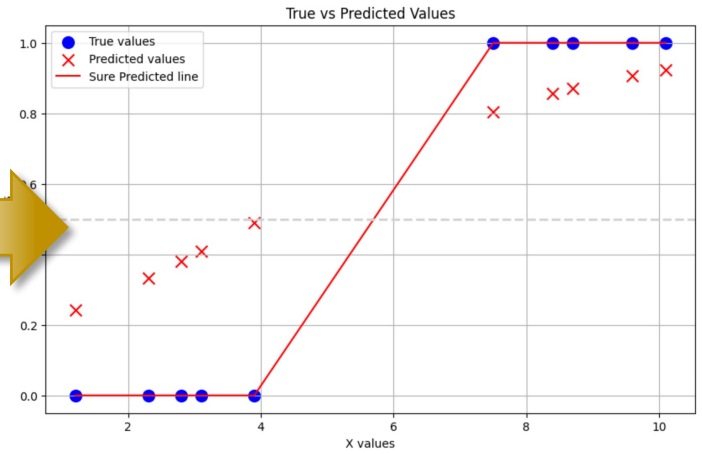
hello world 의 Keras 버전(binary classification)



데이터 정의

```
[ ]  x = [1.2, 2.3, 2.8, 3.1, 3.9, 7.5, 8.4, 8.7, 9.6, 10.1] #입력특성
     y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]#실제 정답
```





예측

```
[25] model.predict([2])

     1/1 [==============================] - 0s 196ms/step
     array([[0.30733138]], dtype=float32)

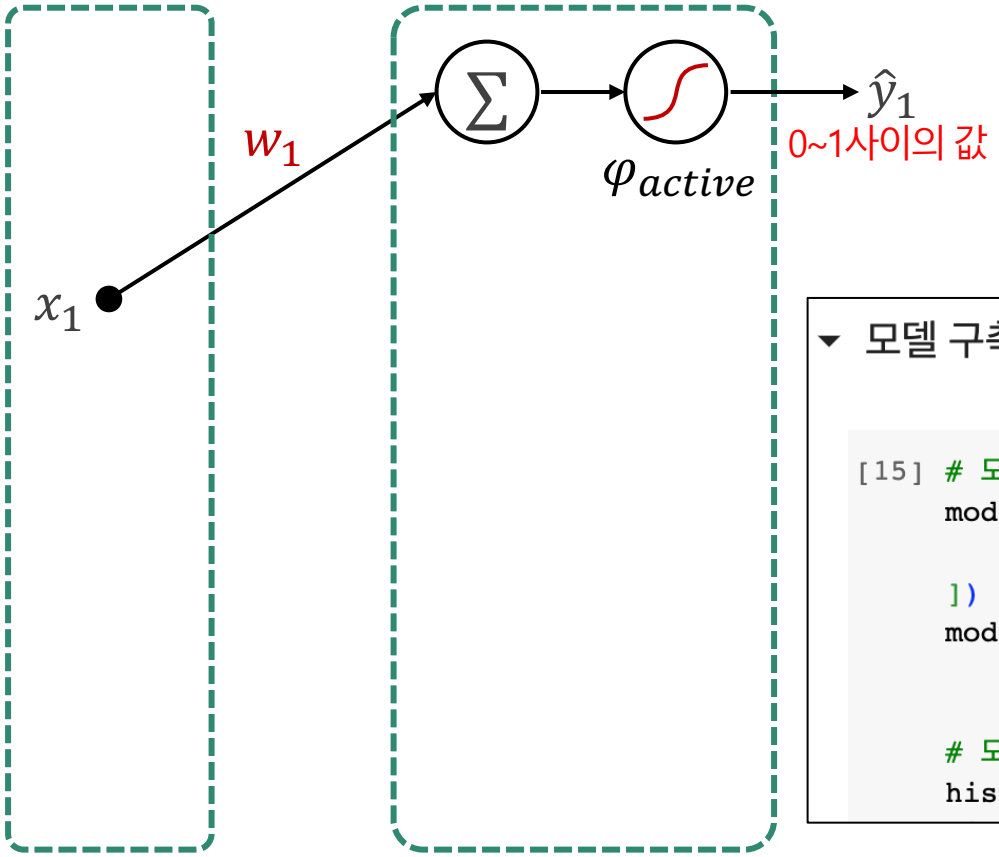
     model.predict([10])

     1/1 [==============================] - 0s 60ms/step
     array([[0.91908693]], dtype=float32)
```

2. 실습 : 텐서플로를 이용한 단층 퍼셉트론

- 구현할 단층 퍼셉트론(binary classification) :

1. 활성화(Activation) 함수 추가
2. 손실(loss) 함수의 경우, Binary Cross Entropy 기법 활용



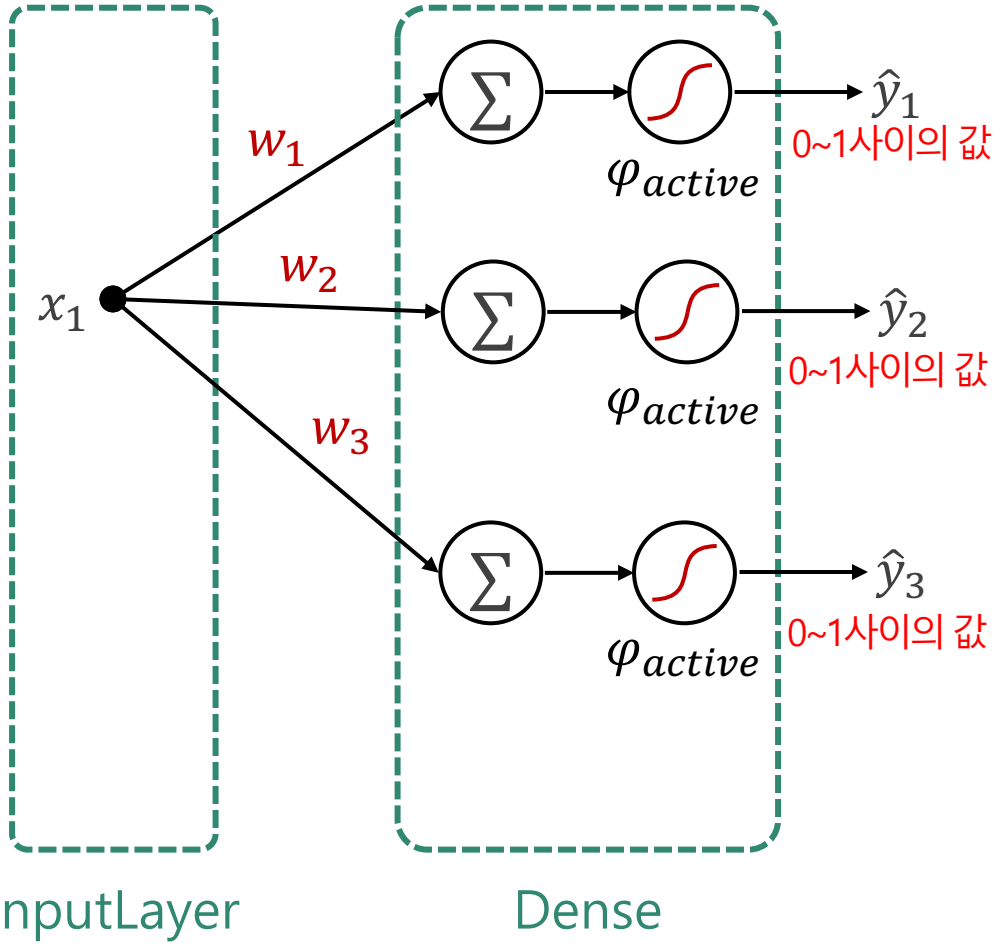
모델 구축 및 학습

```
[15] # 모델 구축(SLP=single_layer_perceptron) -> y_hat = activate(wX+b)
     model = tf.keras.Sequential([
         tf.keras.layers.Dense(units=1, input_shape=(1,), activation='sigmoid'),
     ])
     model.compile(optimizer='sgd', loss='binary_crossentropy')

     # 모델 훈련(train)
     history = model.fit(X, y, epochs=1000)
```

2. 실습 : 텐서플로를 이용한 단층 퍼셉트론

- Q. 구현할 단층 퍼셉트론에서 유닛을 아래와 같이 확장한다면 어디를 고쳐야 될까?



units=1 → units=3

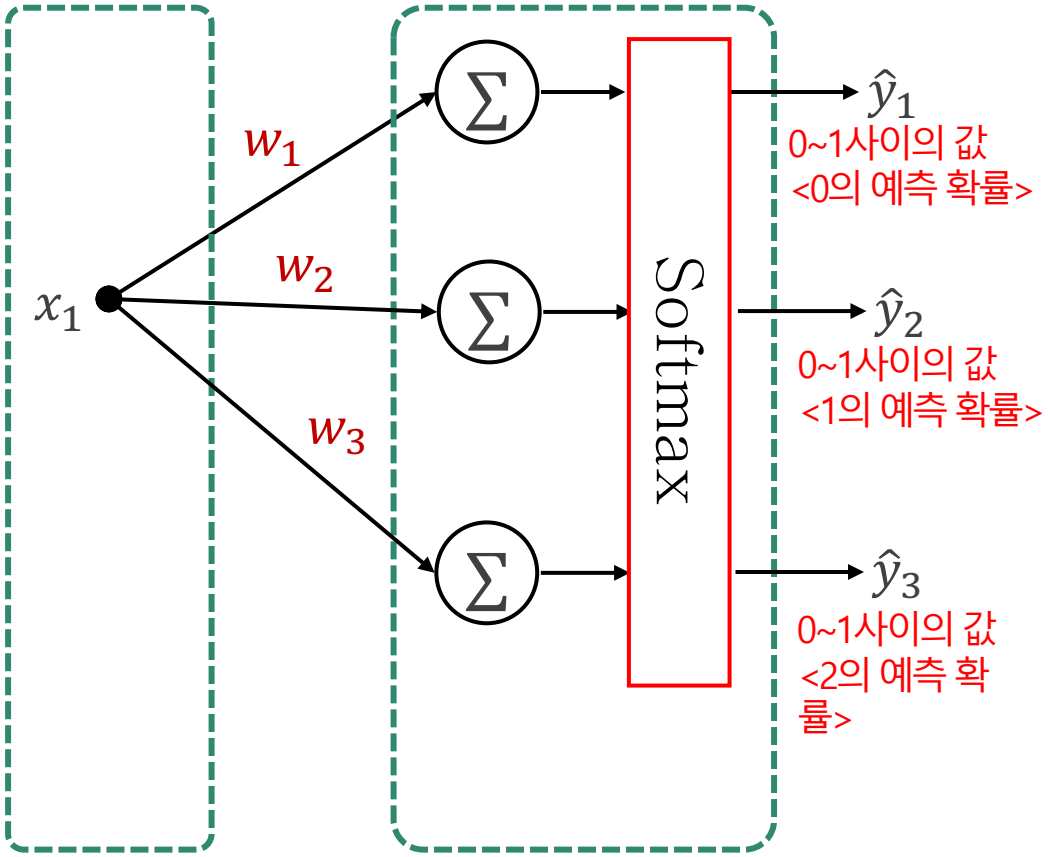
모델 구축 및 학습

```
[15] # 모델 구축(SLP=single_layer_perceptron) -> y_hat = activate(wX+b)
     model = tf.keras.Sequential([
         tf.keras.layers.Dense(units=3, input_shape=(1,), activation='sigmoid'),
     ])
     model.compile(optimizer='sgd', loss='binary_crossentropy')


     # 모델 훈련(train)
     history = model.fit(X, y, epochs=1000)
```

2. 실습 : 텐서플로를 이용한 단층 퍼셉트론

- 구현할 단층 퍼셉트론(유닛 확장) → 출력 확장을 위한 activation 변경(sigmoid -> softmax)



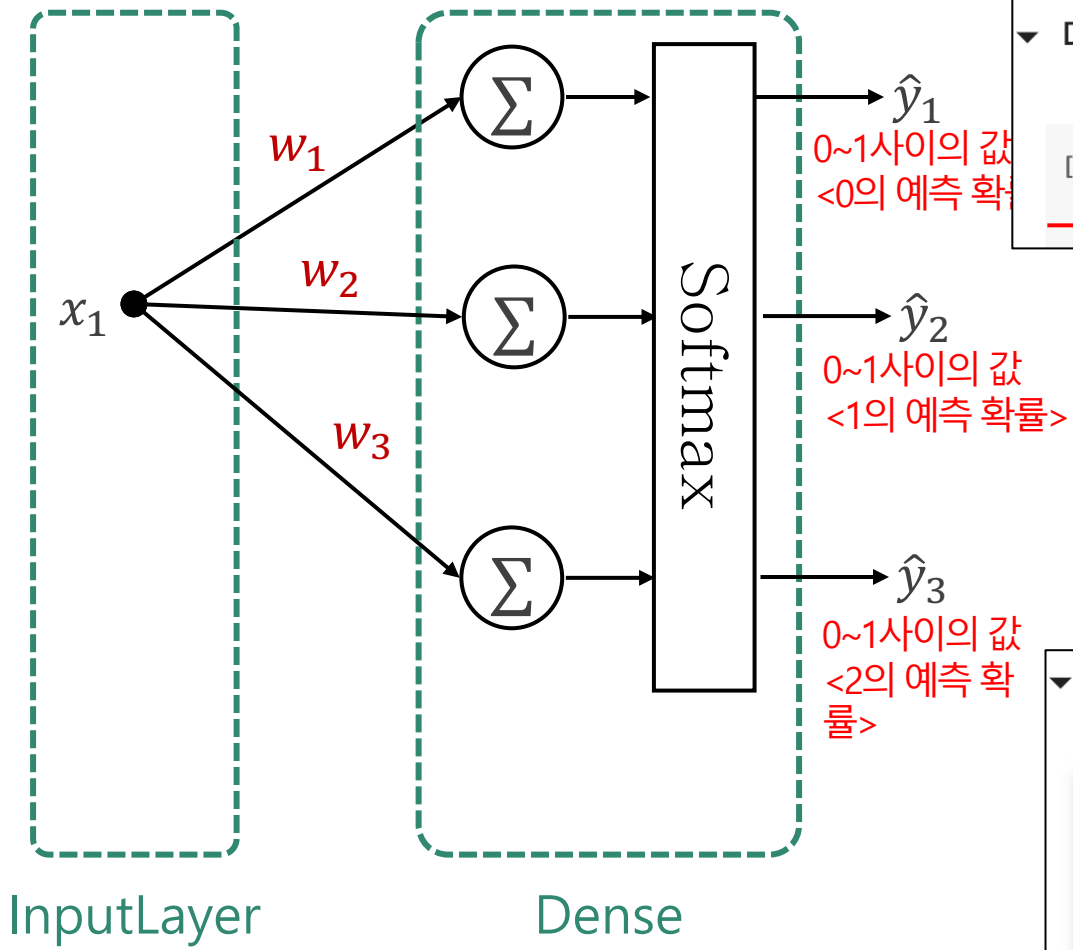
모델 구축 및 학습

```
[15] # 모델 구축(SLP=single_layer_perceptron) -> y_hat = activate(wX+b)
     model = tf.keras.Sequential([
         tf.keras.layers.Dense(unit activation='softmax'
     ])
     model.compile(optimizer='sgd', loss='binary_crossentropy')


     # 모델 훈련(train)
     history = model.fit(X, y, epochs=1000)
```

2. 실습 : 텐서플로를 이용한 단층 퍼셉트론

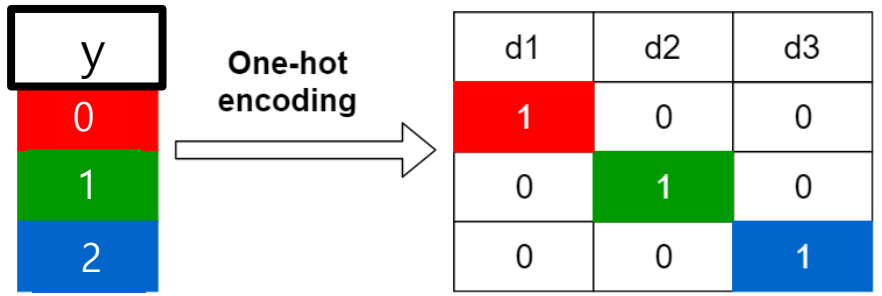
- 구현할 단층 퍼셉트론(유닛 확장) → 예측 값 3개(정답 데이터도 변경 1개씩 → 3개씩)



데이터 정의

```
[ ]  x = [1.2, 2.3, 2.8, 3.1, 3.9, 7.5, 8.4, 8.7, 9.6, 10.1] #입력특성
     y = [0, 0, 0, 1, 1, 1, 2, 2, 2, 2]#실제 정답
```

출력 차원 변경을 위한 OneHot Encoding 활용

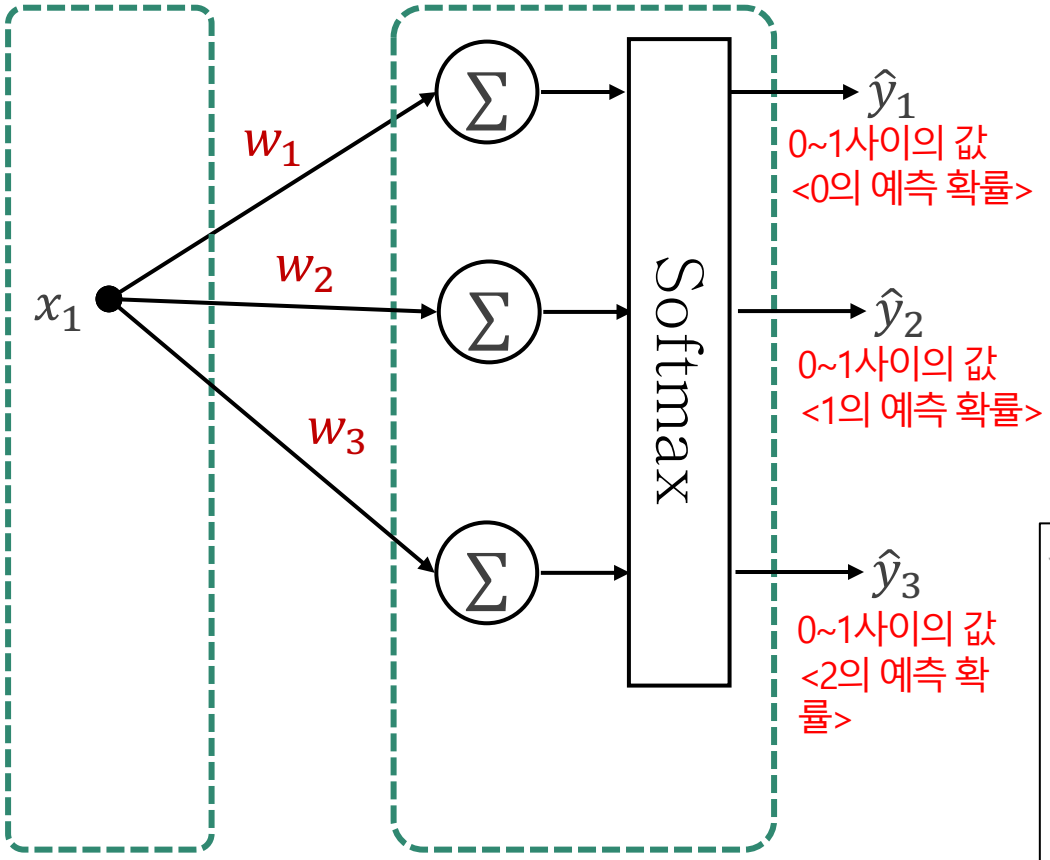


onehot encoding된 데이터 정의

```
x = [1.2, 2.3, 2.8, 3.1, 3.9, 7.5, 8.4, 8.7, 9.6, 10.1] #입력특성
onehot_y = [[1, 0, 0],[1, 0, 0],[1, 0, 0], #onehot 변환된 실제 정답
      [0, 1, 0,],[0, 1, 0],[0, 1, 0],
      [0, 0, 1],[0, 0, 1],[0, 0, 1],[0, 0, 1]]
```

2. 실습 : 텐서플로를 이용한 단층 퍼셉트론

- 구현할 단층 퍼셉트론(유닛 확장) → loss 변경(binary entropy-> categorical entropy)



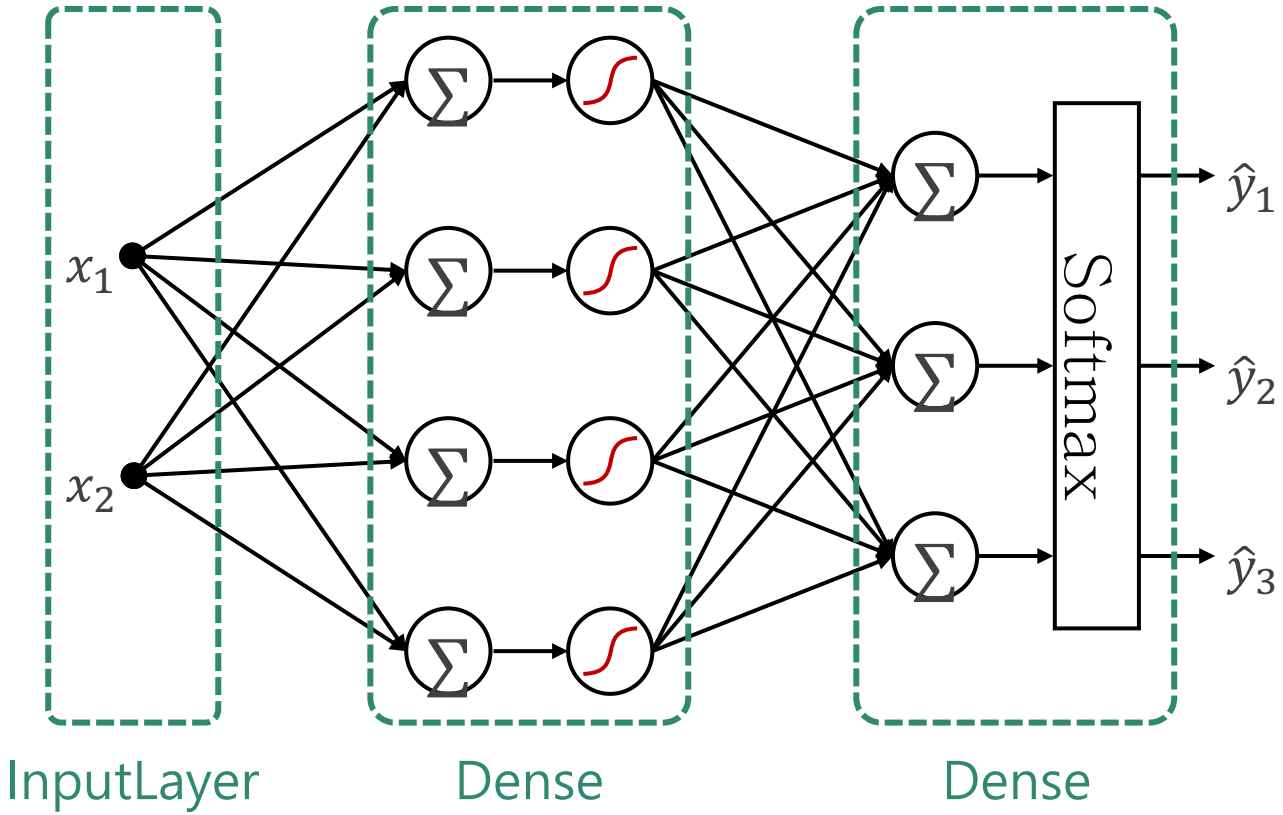
모델 구축 및 학습

```
[15] # 모델 구축(SLP=single_layer_perceptron) -> y_hat = activate(wX+b)
     model = tf.keras.Sequential([
         tf.keras.layers.Dense(units=1, input_shape=(1,), activation='softmax'),
     ])
     model.compile(optimizer='sg', loss='categorical_crossentropy')

     # 모델 훈련(train)
     history = model.fit(X, y, epochs=1000)
```

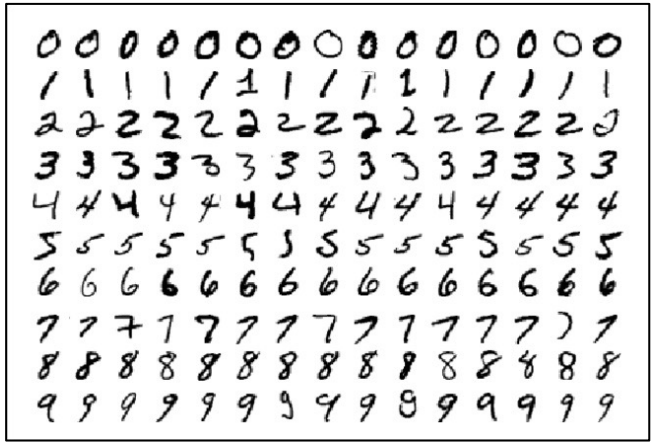
2. 실습 : 텐서플로를 이용한 다층 퍼셉트론

구현할 다층 퍼셉트론



1) Hello world concept 기반,
단층 퍼셉트론을 수정하여 모델 구현해보기
(힌트 : 입력 차원, 히든 레이어 고려)

2) Mnist dataset(0~9 손글자 이미지) 기반,
다층 퍼셉트론 모델 구현해보기



지금부터는

실습을 통해서

진행하겠습니다

https://colab.research.google.com/drive/1SsPtN__UAtU26lUgaXMk0rECLWmND6oM?usp=sharing
(URL 클릭)

딥러닝

수고하셨습니다.



총 점수 :30점

출석 : 15점

과제 : 15점

- 오늘 배운 내용 기반 **딥러닝 과제(객관식+주관식)**